

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ
ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет информатики, математики и компьютерных наук
Программа подготовки бакалавров по направлению
01.03.02 Прикладная математика и информатика

Тюрин Владислав Николаевич

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

Построение графа знаний на основе пользовательских данных с
возможностью экспорта в Obsidian

Научный руководитель
к. ф. – м. н., старший преподаватель

В. М. Кочеганов

Нижний Новгород, 2026

ОГЛАВЛЕНИЕ

Введение	4
1. Теоретические основы и обзор предметной области	7
1.1. Системы управления персональными знаниями	7
1.2. Технологии поиска и обработки естественного языка в системах RAG	10
1.3. Анализ существующих решений для построения графов знаний . . .	13
1.4. Методология оценки качества	19
2. Разработка алгоритмов	25
2.1. Архитектура алгоритма построения графа знаний	25
2.2. Алгоритм GraphRAG	32
3. Сбор и подготовка данных	37
3.1. Поиск данных	37
3.2. Создание валидационного датасета	38
3.2.1 Алгоритм генерации связей	40
3.2.2 Алгоритм извлечения контекста	43
3.2.3 Алгоритм LLM-типизации связей	43
4. Эксперименты	45
4.1. Параметры воспроизводимости	45
4.2. Оценка конвейера построения графа знаний	45
4.2.1 Извлечение кандидатных пар	45
4.2.2 Типизация связей	50
4.3. Сравнение сгенерированных ссылок с эталонными	53
4.4. Оценка конвейера GraphRAG	54
4.4.1 Анализ референсных решений	54
4.4.2 Сравнение собственных алгоритмов с аналогами	56
4.4.3 Влияние промпта и языковой модели	58
4.4.4 Анализ влияния графа.	60

4.5. Выводы и ограничения исследования	61
5. Заключение	63
6. Список литературы	66
7. Приложение А. Визуализация датасета	72

ВВЕДЕНИЕ

В эпоху информационной перегрузки возможность эффективно управлять данными стала особенно важной. Классических способов работы с информацией, таких как ведение заметок, уже недостаточно, особенно для специалистов регулярно работающих с огромными массивами данных из разных областей [1]. В качестве решения были разработаны специализированные системы для управления знаниями, например, Obsidian [2]. Они берут на себя роль «второго мозга» [3], освобождая пользователей от хранения всей информации в голове и упрощая работу с ней.

Современные системы управления знаниями поддерживают генерацию с дополненной выборкой (Retrieval-Augmented Generation, RAG) [4]. Большинство RAG-систем используют гибридный поиск (комбинацию лексического [5] и семантического [6]). Такой подход имеет несколько слабостей: лексический поиск не умеет искать синонимы, а семантический может упустить релевантные данные из-за большой разницы в векторных представлениях (эмбедингах) [7].

Более продвинутой стратегией поиска является использование графов знаний (ГЗ) [8], которые соединяют семантически связанные сущности. Каждая связь имеет направление и тип, отражающие семантическое отношение между сущностями. Это открывает доступ к многошаговому поиску, что решает проблемы гибридного поиска и позволяет увеличить полноту результатов.

Несмотря на преимущества ГЗ, их построение и поддержание требуют колоссальных трудозатрат. По мере роста базы знаний возникает «узкое место»: пользователям недостаточно когнитивных ресурсов и времени на анализ и связывание заметок, из-за чего дальнейшее развитие базы становится невозможным.

Существующие на данный момент решения (графовые базы данных корпоративного уровня [9, 10] или облачные приложения [11]) имеют критические недостатки. Базы данных требуют дорогостоящей инфраструктуры

и чрезмерно зависят от больших языковых моделей (LLM) [12], что приводит к большим издержкам и созданию бессмысленных связей [13]. Облачные решения, несмотря на их удобство для повседневного использования, создают риск утечки данных и насильно привязывают пользователей к экосистеме.

Цель работы — разработка автоматизированного алгоритма для построения графов знаний с помощью семантического связывания заметок в экосистеме Obsidian, а также сравнение разработанного решения с существующими аналогами в сценариях использования RAG.

Для достижения поставленной цели были сформулированы следующие задачи:

1. Провести анализ существующих решений для построения графов знаний для выявления их ограничений и функциональных особенностей.
2. Разработать алгоритм автоматического поиска и типизации связей, включающий: извлечение содержимого Markdown-файлов; гибридный поиск кандидатов; классификацию связей; сохранение найденных отношений, поддерживающее инкрементальные обновления.
3. Создать систему валидации и репрезентативный аннотированный набор данных для объективной оценки качества работы алгоритма и сравнения с аналогами.
4. Провести итеративное улучшение решения, состоящее из экспериментов с различными моделями, гиперпараметрами и методами фильтрации связей, для повышения точности и ее вычислительной эффективности системы.

Объект исследования — процессы поиска, структурирования и извлечения информации в системах управления знаниями. Предмет исследования — методы и алгоритмы автоматического семантического связывания текстовых данных для построения ГЗ.

Использованные в работе методы исследования включают: методы обработки естественного языка (NLP), методы машинного обучения (LLM и

модели эмбедингов), методы информационного поиска (BM25, семантический поиск), элементы теории графов, а также методы системного анализа и эмпирического тестирования.

Научная новизна данной работы заключается в создании автоматизированного алгоритма семантического связывания Markdown-заметок, комбинирующего гибридный поиск, модели машинного обучения (Cross Encoder [14] и LLM), сохранение состояния работы и инкрементальных обновлений ГЗ. В отличие от существующих решений, разработанный алгоритм оптимизирован для работы локально при ограниченных вычислительных мощностях, обеспечивая полный суверенитет данных.

Теоретическая значимость работы состоит в развитии подходов автоматизированного извлечения семантических отношений из неструктурированных текстовых данных, адаптации алгоритма GraphRAG для работы в экосистеме Obsidian, в том числе с типизированными ссылками, а также оценке реализованных решений.

Достоверность полученных результатов подтверждается использованием стандартизированных подходов к оценке качества информационного поиска и алгоритмов RAG, использованием вручную размеченного валидационного набора данных для оценки, а также согласованность полученных практических результатов с теорией.

Алгоритм, разрабатываемый в данной работе, ориентирован на продвинутых пользователей баз знаний, для которых важны суверенность данных, возможность работы алгоритма локально или при ограниченных вычислительных мощностях, а также поддержка эффективного обновления ГЗ. Все вышеперечисленные особенности, реализованные в рамках одной системы, не имеют аналогов, и, следовательно, обладают высокой практической ценностью.

ТЕОРЕТИЧЕСКИЕ ОСНОВЫ И ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Системы управления персональными знаниями

В эру информационной перегрузки необходимо эффективно управлять данными. Традиционные методы хранения и обработки информации, такие как рукописные заметки или их цифровые аналоги, хороши для повседневного использования, но не способны обеспечить необходимую функциональность для специалистов, работающих с огромными массивами информации. Возрастающая актуальность этой проблемы стимулировала развитие специальных систем для управления персональными знаниями (Personal Knowledge Management, PKM).

В основу PKM-систем легли результаты множества исследований из области информационной архитектуры и когнитивной психологии. В последнее время большое влияние оказывает концепция «Второго мозга» (Second Brain), популяризированная исследователем Тиаго Форте [3]. Эта концепция позволяет переложить хранение и организацию информации на внешние носители, тем самым разгружая рабочую память человека.

Человеческое мышление ассоциативно, в нем одна концепция может принадлежать разным областям знания. Иерархические структуры не обладают достаточной гибкостью, так как требуют однозначной категоризации, что противоречит устройству человеческого мышления. Их все чаще заменяют на сетевые структуры, наподобие Zettelkasten («картотечный ящик») [1]. Этот подход, разработанный социологом Никласом Луманом, заключается в хранении небольших атомарных заметок, связанных друг с другом. Связи в нем представляют основную ценность: информация, хранящаяся в заметках, не абстрагирована, а встроена в общий пласт знаний, что ускоряет поиск и позволяет делать сложные, на первый взгляд неочевидные умозаключения.

Популярным представителем современной реализации подхода Zettelkasten является программа Obsidian [2]. В ней заметки хранятся в виде файлов формата Markdown, которые пользователь может хранить в удобном ему месте. Заметки

поддерживают двунаправленное связывание путем упоминания заметки в тексте (формат [[wikilink]]). Кроме того, в заметках можно добавлять специальные пометки, теги, по которым можно осуществлять быструю фильтрацию и поиск. Такая архитектура облегчает организацию данных, а также обеспечивает их суверенитет, поскольку пользователь сам выбирает место их хранения.

В основе сетевых РKM-систем лежит концепция персонального графа знаний. Граф знаний представляет собой структуру, объединяющую информацию о сущностях и взаимосвязи между ними. Его узлы — это сущности, например, объекты, люди, термины, идеи и так далее. Ребра — направленные типизированные связи.

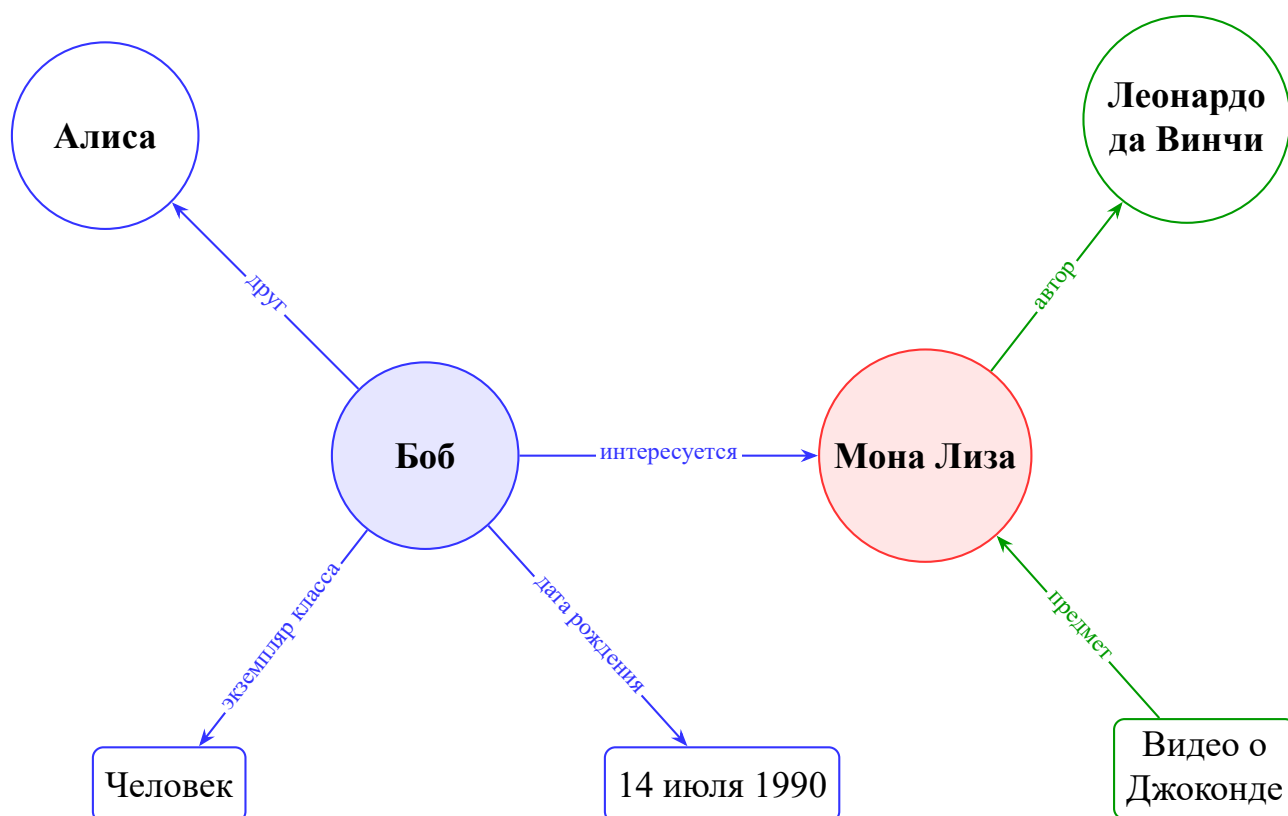


Рис. 1 – Пример графа знаний

Подобная структура данных позволяет создавать кластеры (сообщества) из семантически близких сущностей, осуществлять поиск на основе связей. Это имеет особую ценность в системах управления знаниями, так как превращает разрозненные пользовательские заметки в связанные группы, чем делает поиск и работу с данными эффективнее и результативнее.

Простые и семантически типизированные связи

В ГЗ все ребра имеют направление и тип, который отражает характер отношения между сущностями. Это обеспечивает более продвинутую навигацию, позволяя отсеивать нерелевантные направления на раннем этапе, что открывает возможность использовать многошаговые запросы.

В механизме wikilink-ссылок, который использует Obsidian, по умолчанию нет поддержки типизации. Однако она может быть добавлена с помощью популярного расширения DataView [15], которое позволяет задавать тип связи при помощи следующего синтаксиса: `link_type:: [[wikilink]]`.

Специфика целевой аудитории

Основной аудиторией систем управления знаниями являются люди, чья деятельность тесно сопряжена с обработкой огромных массивов данных. Исследователи, журналисты и другие специалисты используют РKM системы для агрегации данных, создавая краткие выжимки, для их трансформации, находя пробелы в знаниях или данных, и множества других целей. Например, эти системы могут быть использованы студентами для подготовки ответов на экзаменационные вопросы на основе материалов занятий.

Постановка проблемы: Когнитивное «узкое место»

У ГЗ есть недостаток, заключающийся в ручном процессе извлечения и связывания информации. На начальных этапах создание ссылок и их типизация происходит естественным образом и не требует особых усилий. Однако при преодолении определенного порога по объему информации, например, при сотнях или тысячах заметок в хранилище пользователь утрачивает возможность удерживать весь контекст в голове. Из-за этого процесс связывания становится трудозатратным и неточным: пользователь начинает пропускать связи, особенно неочевидные. Как следствие, граф становится разреженным и деградирует поиск, так как из-за отсутствующих связей алгоритму не удастся найти необходимую

информацию. В итоге граф начинает терять свою ценность.

Таким образом, алгоритм, способный автоматически анализировать семантику заметок, находить связи между ними и типизировать их, тем самым поддерживая актуальность и плотность графа знаний, устранил это «узкое место» и освободит когнитивные ресурсы пользователей под другие задачи.

1.2 Технологии поиска и обработки естественного языка в системах RAG

Использование генеративных моделей в чистом виде влечет высокие риски «конфабуляций» и «галлюцинаций»: ответ модели может казаться правдоподобным, но содержать фактологические или причинно-следственные ошибки [13].

Для решения этой проблемы была разработана технология Retrieval-Augmented Generation (RAG) [4]. Она добавляет дополнительный этап поиска информации перед генерацией ответа. На основе запроса пользователя алгоритм извлекает наиболее релевантную информацию из доступных источников, таких как текстовые файлы или базы данных, после чего передает ее языковой модели в качестве контекста. Такой подход заставляет LLM выступать в роли анализатора, а не источника фактов.

Алгоритмы поиска

Эффективность RAG-систем зависит от качества алгоритмов извлечения. В связи с этим существует несколько подходов к поиску необходимых данных. Наиболее популярными методами являются лексический и семантический поиск.

Лексический поиск основан на точных совпадениях лексем из запроса пользователя с содержащимися в хранилищах.

Современные реализации этого подхода, такие как BM25 [5], используют вероятностные модели и метрику TF-IDF. Такие решения оценивают частоту встречаемости терминов в файлах и штрафуют, если частота во всем корпусе слишком высока. Это придает больший вес редко используемым терминам, формулировкам, именам собственным. Результатом является отсортированный по

релевантности список документов или заранее выделенных из них фрагментов.

Данный подход отлично справляется с поиском специфических терминов и точных формулировок, но испытывает трудности, например, в случае замены слов на синонимы.

Для преодоления этих ограничений используется семантический поиск, основанный на векторном сходстве [6]. Исходный текст разбивается на фрагменты (чанки), для каждого из которых вычисляется многомерный числовой вектор (эмбединг) из одного пространства. Поиск происходит путем вычисления сходства между этими векторами с помощью метрик, таких как косинусное сходство. Если полученное значение сходства достаточно велико, то чанки считаются релевантными, и текст, соответствующий эмбедингу, извлекается. Этот подход позволяет улавливать смысл текста, не основываясь на конкретных словах или формулировках.

Векторный поиск также имеет несколько недостатков [7]:

- Потеря конкретики: смысловое значение фрагментов сжимается в вектор фиксированной размерности, что ведет к размытию семантики и потере деталей.
- Проблема доменного сдвига и асимметрии запроса: сильное различие в размере текстовых фрагментов может повлечь изменение семантического распределения, в результате чего векторы, полученные на основе релевантных данных, могут оказаться на большом расстоянии друг от друга, и, как следствие, могут быть не найдены.

Современные RAG системы комбинируют эти подходы в так называемый гибридный поиск [7]. Он позволяет компенсировать недостатки одного подхода с помощью другого, что потенциально может увеличить полноту поиска. В нем извлеченные чанки получают унифицированную оценку и ранжируются по ней.

Переранжирование поисковой выдачи

Однако при использовании вышеупомянутых алгоритмов выдача может содержать ложноположительные результаты, в которых искомые термины используются в ином от запроса контексте. Для дополнительной фильтрации результатов применяются нейросетевые модели архитектуры Cross-Encoder [14]. Данные модели принимают два текста (в данном случае запрос пользователя и текст заметки), после чего обрабатывают их через слои перекрестного внимания (cross-attention). Такой подход позволяет оценить тексты на наличие логических взаимосвязей, тем самым точнее отфильтровать нерелевантные фрагменты.

GraphRAG

Несмотря на все преимущества этих подходов, обработка сложных аналитических запросов, требующих многошагового рассуждения, вынуждает использовать более сложные алгоритмы поиска. В случаях, когда нужная информация напрямую не связана с запросом, она не может быть найдена ни семантическим, ни лексическим поиском, из-за чего поисковый алгоритм RAG не сможет извлечь необходимые данные. Эта проблема остро стоит в системах управления персональными знаниями, в которых информация организована по принципу Zettelkasten.

Более продвинутой стратегией поиска, решающей ограничения изолированного поиска, является использование графов знаний для расширения контекста (Knowledge Graph-augmented RAG, GraphRAG) [16]. В этом подходе вышеописанные алгоритмы поиска дополняются обходом ГЗ. Граф, используемый в поиске, как правило заранее построен на основе пользовательских данных. Такой подход позволяет не только собрать контекст из вершин, найденных первоначальным поиском, но и дополнить его путем обхода семантически связанных с ними узлов и сбора их содержимого.

Конкретной реализацией, давшей существенное развитие GraphRAG, является архитектура, предложенная исследователями из Microsoft [8]. Их

решение добавляет дополнительный этап при построении графа, заключающийся в поиске сообществ. На базовом графе используются алгоритмы кластеризации — граф разбивается на плотные подграфы (сообщества), для каждого из которых с помощью LLM генерируется описание. Созданные сообщества позволяют предоставить системе поиска глобальное понимание графа, а также эффективно обрабатывать общие запросы, ответ на которые требует агрегации информации из нескольких заметок.

1.3 Анализ существующих решений для построения графов знаний

В настоящее время на рынке представлено множество решений для управления знаниями, которые автоматизированно строят ГЗ. Они варьируются от полноценных экосистем, предназначенных для корпоративного использования, до узконаправленных инструментов, решающих конкретные задачи. Проведенный анализ позволил разделить решения на три группы, выявить ограничения и хорошие практики существующих решений, а также сформировать базовый набор технологий, применяемый при построении ГЗ в контексте систем управления знаниями.

Готовые решения для корпоративного использования.

В данную категорию включены полноценные экосистемы, в которых есть возможность автоматически трансформировать пользовательские текстовые данные в ГЗ. Представителями этой категории являются такие решения как Neo4j Graph Builder [9] и Memgraph Unstructured2Graph [10]. Neo4j и Memgraph являются графовыми базами данных, нацеленными на эффективное хранение данных и обработку запросов. Однако они имеют специальные модули, предоставляющие функционал для построения ГЗ в этих базах на основе текстовых данных.

Поскольку данная работа фокусируется на построении ГЗ, в этих решениях будет изучен исключительно ответственный за это модуль. Оба решения имеют открытый исходный код этих модулей. За основу взят пайплайн Neo4j, однако

логика Memgraph крайне схожа. Изученный конвейер состоит из нескольких этапов:

1. Парсинг и чанкинг. Исходные данные разбиваются на фрагменты размером 4000 символов и перекрытием 200 символов [9].
2. Извлечение сущностей. Каждый чанк передается в LLM с инструкциями для извлечения троек (Entity A – [relation type] – Entity B).
3. Разрешение кореференций. Извлеченные имена сущностей дедуплицируются с помощью большой языковой модели во избежания конфликтов (например, сущности «ИИ» и «Искусственный интеллект» объединяются в одну с именем «Искусственный интеллект»).
4. Запись данных. Обработанные данные записываются в базу данных. Они формируют два графа: структурный, сохраняющий последовательность фрагментов в файле, и семантический, содержащий связи между сущностями.
5. Создание сообществ. Для нахождения групп из близких по смыслу вершин используется алгоритм Лувена. Для каждой группы создается вершина, описание которой состоит из агрегированного содержимого заметок группы.

Neo4j и Memgraph используют LangChain [17] для работы с LLM, что позволяет использовать множество провайдеров, а также запущенные локально модели.

Данные системы обладают рядом преимуществ:

1. Поддержка множества форматов текстовых данных, таких как PDF, HTML и DOCX.
2. Построение двухуровневых графов, упрощающих сбор всего содержимого одной заметки, например, в случае необходимости пересказа ее содержимого

3. Формирование сообществ с описаниями, что экономит объем контекстного окна при ответе на общие вопросы, затрагивающие множество заметок на общую тему.

Однако они также имеют ряд критических недостатков, ставящих под сомнение использование подобных решений для персональных баз данных.

1. Отсутствие поддержки частичного обновления. Даже при минимальных модификациях файлов необходимо полностью перезапускать дорогостоящий пайплайн для построения ГЗ.
2. Разрыв между данными и графом. Поскольку в этих сервисах отсутствуют механизмы синхронизации содержимого заметок, пользователю необходимо вручную загружать обновленные файлы на сервер базы данных, что увеличивает вероятность ошибок и делает процесс более трудоемким.
3. Инфраструктурные издержки. Для обработки данных и взаимодействия с ГЗ необходима работающая база данных, что ведет к потреблению дополнительных вычислительных ресурсов, и, как следствие, дополнительным затратам, если сервис работает на выделенном сервере.
4. Чрезмерное доверие к LLM. В пайплайнах этих решений отсутствует фильтрация кандидатов. В них вся работа по извлечению сущностей и анализу связей полностью лежит на больших языковых моделях, которым свойственны «галлюцинации». Это влечет создание бессмысленных и выдуманных связей, которые делают извлечение информации неэффективным.

Академические автоматизированные пайплайны.

В качестве альтернативы корпоративным платформам, опирающимся на использование тяжеловесных LLM, было найдено решение [18], эффективно использующее сравнительно небольшие модели и другие методики для построения ГЗ.

Его пайплайн состоит из следующих этапов:

1. Чанкинг. Файлы разбиваются на фрагменты, каждый из которых будет обрабатываться независимо, то есть для каждого чанка будет создан независимый ГЗ.
2. Извлечение сущностей. Вместо больших языковых моделей данное решение использует компактную модель SqueezeBERT [19]. Она формирует контекстно-зависимые векторные представления для каждого токена, что позволяет вычислить значимость использованных терминов и извлечь главные, которые впоследствии будут вершинами графа.
3. Привязка к онтологиям. Для стандартизации имен сущностей и снижения галлюцинаций все извлеченные сущности сверяются с базами данных, такими как DBpedia или Wikidata.
4. Вычисление семантического сходства. Для создания связей между сущностями используется сравнение косинусного сходства их плотных эмбедингов, созданных моделью Sentence-BERT [20], с пороговым значением.
5. Слияние графов. Все ГЗ, построенные для фрагментов текстов, объединяются в единую сеть на основе общих узлов и связей.

Ключевые достоинства этого подхода включают:

1. Вычислительную эффективность. Использование небольших моделей машинного обучения сильно экономит вычислительные ресурсы и позволяет использовать решение полностью локально.
2. Использование онтологий. Это решение упрощает этап разрешения кореференций, уменьшая вероятность галлюцинаций и убирая необходимость использования LLM для дедупликации.

У данного решения есть несколько недостатков, делающих этот подход неподходящим для решения главной проблемы этой работы.

1. Обязательное участие человека. Поскольку авторам важно качество полученного ГЗ, сгенерированный граф тщательно корректируется человеком, что делает процесс лишь частично автоматизированным.
2. Малое разнообразие сущностей. Использование жестких списков допустимых сущностей может повлечь ошибочную фильтрацию непопулярных терминов или аббревиатур.

Облачные приложения для заметок.

Индустрия также имеет примеры приложений для ведения заметок, которым удалось интегрировать ГЗ для увеличения эффективности их использования.

Облачные сервисы, такие как Mem.ai [11], являются программным обеспечением как услугой (SaaS), полностью разгружая пользователя от организации и связывания заметок. Mem.ai, по аналогии с MemGraph и Neo4j, строит двухуровневый граф (структурный слой, сохраняющий последовательность фрагментов в файле, и семантический, содержащий связи между сущностями). Это обеспечивает эффективную навигацию алгоритмам RAG. В данном сервисе ГЗ используются как для подсказок во время написания заметок, так и для извлечения и синтеза информации посредством интегрированного чат-бота.

Несмотря на все удобства, у Mem.ai существуют недочеты, касающиеся приватности и обработки данных:

1. Закрытый исходный код. Пользователи не имеют доступа к логике построения ГЗ и обработки их заметок.
2. Привязка к поставщику. Данные хранятся на серверах компании, что ставит под сомнение их безопасность, и делает их экспорт в случае закрытия сервиса

затруднительным: сервис предоставляет возможность экспорта сырого текста заметок, но не ГЗ.

Плагины Obsidian.

Приложение имеет поддержку пользовательских расширений, которые добавляют новый функционал в программу. По результатам исследования выяснилось, что расширений, позволяющих автоматизировать процесс построения ГЗ, не выявлено. Несмотря на это, есть плагины-ассистенты, упрощающие некоторые его этапы.

1. Smart Connections [21]. Плагин, использующий векторный поиск для формирования списка схожих заметок. Инструмент способен находить только семантически близкие заметки. Кроме того, плагин не проводит никакой дополнительной фильтрации результатов, а также не типизирует результаты, чем перекладывает большую часть работы на пользователя.
2. Graph Analysis [22]. Инструмент, подсказывающий связи на основе существующих. Данное решение опирается только на уже построенный граф для вычисления статистик, на основе которых предлагает пользователю новые связи или хабы. Данный плагин может хорошо работать как дополнение к автоматизированному пайплайну построения ГЗ, но не как его замена.

Выводы.

Решения, представленные на рынке в данный момент, заставляют пользователей идти на компромиссы. С одной стороны, есть либо полноценные дорогостоящие и требовательные к ресурсам базы данных, либо закрытые облачные решения для ведения заметок, которые все же решают задачу автоматизированного построения ГЗ. С другой стороны, существуют открытые легковесные инструменты, позволяющие упростить некоторые этапы построения, но требующие вмешательства человека.

Решение с открытым исходным кодом, способное вычислительно эффективно находить, фильтровать, типизировать и сохранять связи, а также поддерживающее работу полностью локально, имело бы огромное преимущество на фоне изученных решений. Плюсом будет возможность модификации построенного графа пользователем, как и поддержка инкрементальных обновлений.

1.4 Методология оценки качества

Для объективной оценки разрабатываемого пайплайна и его сравнения с аналогами необходимо создать методологию тестирования. Процесс тестирования требует как подготовки валидационного датасета, так и выбора подходящих метрик.

Реализация графа знаний в среде Obsidian и требования к данным

Благодаря гибкости Obsidian и нативной поддержке ссылок, создание ГЗ в нем будет выглядеть естественно. В качестве сущностей будут выступать Markdown-файлы, а в роли семантических связей — гиперссылки. В данной работе будут также рассматриваться типизированные ссылки в Obsidian, которые добавляет расширение Dataview.

Также необходимо установить ряд ограничений для сохранения каноничности графа, а также упрощения работы с ним.

1. Корпус текстов должен обладать высокой семантической связностью, то есть между файлами должны быть ярко выраженные семантические отношения (иерархические, причинно-следственные, функциональные и другие).
2. Содержание каждого файла должно быть целостным и описательным.
3. Атомарность сущностей: заметка должна представлять одну сущность.
4. Пространство допустимых сущностей задается файлами в хранилище. При этом создание рефлексивных связей (когда заметка ссылается сама на себя) не допускается.

Раскроем третье структурное ограничение подробнее. Оно означает, что в содержании файла должна находиться информация только об одной конкретной сущности, такой как личность, термин, событие, и так далее. Содержание файла может включать в себя описание этой сущности, ее определение, историю, факты, а также описание связей с другими сущностями. Во-первых, такой подход делает хранилище максимально похожим на граф знаний. Во-вторых, это убирает необходимость извлекать сущности, так как файлы в хранилище уже формируют список допустимых сущностей. В-третьих, данное ограничение соответствует концепции атомарности заметок в Zettelkasten [1], где одна заметка содержит в себе ровно одну идею или концепцию.

Стратегия валидации

Оценка такой комплексной системы от начала до конца является некорректной, так как она может скрывать ошибки промежуточных этапов. Например, если на этапе извлечения были получены нерелевантные чанки, то этап типизации заведомо провалится, так как он не получит нужных данных. В связи с этим валидация конвейера будет разделена на три этапа: оценку извлечения контекста, оценку типизации отношений, а также оценку сгенерированного графа для использования в RAG-системах. Такой подход даст возможность изолированного измерения качества работы каждого модуля системы и его тонкой настройки, что положительно скажется на итоговом качестве пайплайна.

Оценка извлечения контекста

Оценку этапа извлечения (retrieval) можно сформулировать как задачу воспроизведения контекстов, в которых упоминается сущность: для каждой пары (s, e) , где s — файл-источник (source_file), а e — целевая сущность (entity), система должна вернуть фрагмент из текста s , в достаточной мере совпадающий с эталонным контекстом из «золотого стандарта». Извлеченный фрагмент должен полностью содержать эталонное предложение.

Важно заметить, что при больших чанках вероятность случайно захватить

нужное предложение выше. Однако в реальных сценариях извлекаемые фрагменты не могут быть слишком большими, так как это приведет к размытию смыслов и потере деталей, что негативно повлияет на качество извлечения.

Извлеченные фрагменты текста подразделяются на две категории:

- Прямое (явное) упоминание. В данную группу входят все предложения, которые содержат полное или общепринятое краткое наименование. Например, предложение «Билл Гейтс соосновал Microsoft» явно содержит название компании.
- Кореферентное упоминание. Этот класс включает случаи, в которых прямое имя сущности заменено на другие языковые маркеры, требующие контекстного разрешения. Например, в тексте «Microsoft была основана 4 апреля 1975. Билл Гейтс был одним из тех, кто ее соосновал» местоимение «ее» отсылает к Microsoft.

Для оценки необходимо создать специальный датасет, состоящий из предложений с упоминаниями сущностей, списка упомянутых сущностей, а также типа упоминания.

Оценка LLM-типизации

Оценка типизации связей сводится к задаче классификации направленного отношения для упорядоченной пары сущностей (e_1, e_2) . При условии, что этап извлечения предоставил набор релевантных фрагментов, большая языковая модель должна определить наличие связи от e_1 к e_2 , и присвоить ей один из представленных классов. В случае присвоения недопустимого класса или ответа модели о несвязанности сущностей (классификации как отдельный класс `no_relation`), ссылка отбрасывается.

Для оценки качества классификации используются стандартные метрики [23]:

- Точность (Precision) — доля истинных связей среди всех предсказанных

алгоритмом. Метрика важна для оценивания алгоритма на устойчивость к шуму. В случае создания множества ложноположительных связей граф засорится и потеряет свою ценность.

- Полнота (Recall) — доля истинных связей из датасета, которые алгоритм смог успешно идентифицировать и типизировать.
- Взвешенная F-мера (Weighted F_1 -score) — гармоническое среднее между точностью и полнотой. Эта метрика устойчива к дисбалансу в типах связей: зачастую в данных универсальные классы (такие как упоминания или дополнения) встречаются значительно чаще, чем, например, опровержение или противопоставление. Она будет использоваться в качестве основной оценки.

Полная оценка GraphRAG

Для комплексной оценки ответов, сгенерированных LLM на основе контекста, предоставленного RAG, используются специализированные фреймворки, такие как RAGAS [24]. Их фундаментальной идеей является использование LLM как судей для оценки ответов других языковых моделей, что позволяет оценивать семантику ответов, а не использованные в них слова.

В данной работе будет использован фреймворк RAGAS. Из метрик, предоставляемых им, будут подсчитываться:

- Точность извлеченного контекста (Context Precision, CP). Данная метрика оценивает способность RAG систем извлекать релевантную информацию и ранжировать ее по мере соответствия. В RAGAS каждый извлеченный фрагмент текста оценивается на наличие и долю в нем нужной для ответа информации. Высокое значение CP говорит о предоставлении языковой модели релевантных данных с минимальной долей шума в них.
- Полнота извлеченного контекста (Context Recall, CR). Эта метрика оценивает RAG на умение собирать всю необходимую для ответа информацию.

Для расчета этой метрики эталонный ответ разбивается на независимые атомарные утверждения, каждое из которых оценивается на возможность вывода его на основе извлеченных данных. Чем больше утверждений в эталонном ответе могут быть выведены, тем выше значение метрики.

- Достоверность ответа (Answer Faithfulness, F) направлена на измерение достоверности ответа, то есть степени, в которой сгенерированный ответ опирается исключительно на предоставленный контекст, а не на знания самой модели. Процесс оценки заключается в выделении всех тезисов в ответе и проверке возможности их подтверждения на основе предоставленных фрагментов текстов. Если модель использовала в ответе внешнюю информацию, то мера достоверности снижается.
- Корректность ответа (Answer Correctness, AC) измеряет семантическое и фактологическое соответствие сгенерированного ответа эталонному. Во фреймворке RAGAS она вычисляется как взвешенная сумма фактологического перекрытия (доля фактов, присутствующих как в ответе системы, так и в эталоне) и семантического сходства эмбедингов двух ответов.

Использование такого набора метрик позволяет получить полную картину работы пайплайна: от способности алгоритма GraphRAG предоставить релевантную информацию, до способности модели использовать предоставленную информацию для генерации ответа.

Моделирование сложных сценариев для тестирования

Чтобы проверить устойчивость алгоритма и его применимость в реальных условиях использования, необходимо выявить наиболее сложные или важные ситуации, с которыми встречаются алгоритмы построения ГЗ и GraphRAG.

- Обнаружение сообществ. Сообщества — подграфы, вершины внутри которых плотно связаны между собой, но слабо с внешними. Они

формируются из сущностей из одной тематики. Например, заметки с рецептами создают один кластер, а конспекты лекций по машинному обучению — другой. Современные алгоритмы ГЗ обычно автоматически их обнаруживают и создают для них отдельную вершину, в которой содержится краткая сводка [8]. Эти вершины полезны при ответе на широкие вопросы, такие как «Какие темы из области математического анализа я изучал в этом месяце?», а также для навигации алгоритмов поиска релевантных заметок. Они убирают нужду каждый запрос проходить по всем вершинам и обобщать их содержимое. Поскольку алгоритм, представленный в этой работе, не будет искать и создавать сообщества, уместно оценить возможность находить уже созданные. В рамках валидационного датасета вершины-сообщества будут созданы вручную.

- Проверка на устойчивость к шуму. В данном случае можно использовать омонимию, то есть слова или термины, совпадающие по написанию, но различные по смыслу. Такая структура заставит алгоритм использовать содержимое заметок для поиска и фильтрации. Также возможно создание графа из тесно связанных сущностей, что позволит протестировать способность алгоритма улавливать тонкие различия между типами связей в однородном тексте.
- Многошаговый поиск. Линейные цепочки из заметок, где каждая вытекает из предыдущей, проверяют эффективность поиска длинных логических путей.

РАЗРАБОТКА АЛГОРИТМОВ

2.1 Архитектура алгоритма построения графа знаний

Разработанная система является конвейером для автоматического нахождения, типизации и инкрементального сохранения семантических связей между заметками (Markdown-файлами) в хранилище Obsidian. Процесс построения состоит из нескольких шагов: от сканирования хранилища до записи найденных отношений в файлы. Алгоритм поддерживает сохранение состояния файлов в директории и контрольные точки (checkpoints), которые позволяют алгоритму продолжить работу с момента прерывания и нацелены на сохранение вычислительных ресурсов. Алгоритм реализован на языке Python (исходный код доступен в папке `src/kg_builder`).

Общая схема конвейера представлена на рисунке 2.



Рис. 2 – Общая схема конвейера построения графа знаний

Этап 1. Сканирование хранилища и обнаружение изменений (VaultManager)

На этом этапе модуль `VaultManager` рекурсивно обходит директорию хранилища, формируя список всех `markdown`-файлов. Для каждого файла вычисляется `SHA-256` хеш-сумма его содержимого для идентификации изменений. Полученный хэш сравнивается с сохраненным в метаданных (`metadata.json`), с предыдущего запуска. В результате файлы разбиваются на три категории: новые (отсутствующие в метаданных), измененные (не совпадает хеш) и удаленные (есть в метаданных, но отсутствует на диске). Если метаданных нет, то все файлы считаются новыми.

Данный подход позволяет экономить вычислительные ресурсы, поскольку позволяет обрабатывать только измененные файлы, а не все хранилище целиком.

Этап 2. Разбивка на чанки и индексация (VectorStore / LanceDB).

Каждый новый или измененный файл проходит процесс индексации. Эта процедура состоит из двух частей: обработки содержимого заметок и подготовки индексов.

Изначально текст каждого Markdown-файла состоит из двух частей: основного содержимого и блока ссылок, отделенного заголовком `link_header`. Индексируется только основной текст заметки.

Текстовый индекс строится на основе лемматизированного содержимого файлов. Лемматизация проводится при помощи модели `ru_core_news_sm` или `en_core_web_sm` из `spaCy` [25]. Из текстов извлекаются леммы всех алфавитных и числовых токенов в нижнем регистре. Преобразованные тексты передаются в LanceDB [26], где на них строится FTS-индекс, использующий алгоритм BM25. Данный подход позволяет увеличить точность поиска, нивелируя проблему, связанную с изменением форм слов, в которых, например, могут быть изменены окончания или беглые гласные в корне, что особенно распространено в русском языке. Стоит отметить, что вместе с преобразованным текстом сохраняется и оригинальный.

Построение векторного индекса начинается с разбиения содержимого файла на перекрывающиеся чанки. Используется рекурсивное посимвольное разбиение (`RecursiveCharacterTextSplitter` из `LangChain`). Данная стратегия разбивает текст иерархически по заданным разделителям, расположенным в порядке их приоритета, (символ новой строки, точка и т.д.), при этом не превышая заданный размер чанка (по умолчанию 3,000 символов). Эта стратегия сохраняет целостность предложений и структуру Markdown-файлов, что крайне важно для вычисления эмбеддингов и формирования контекста для LLM.

Далее для каждого чанка следует вычисление плотного векторного эмбеддинга при помощи модели из `Sentence-Transformers` [27], например популярной мультиязычной модели `BAAI/bge-m3`.

Полученные индексы объединяются друг с другом, а также с метаданным в

виде пути к файлу и хеша, после чего сохраняются в базу данных. Хранение обоих форматов данных в одной таблице позволяет использовать гибридный поиск. LanceDB позволяет регулировать баланс между векторным и полнотекстовым поиском при помощи параметра `vector_weight` (где `vector_weight` $\in [0, 1]$): при `vector_weight` = 1.0 используется только ANN-поиск, при `vector_weight` = 0.0 — только BM25, промежуточные значения используют оба поиска, сортируя результаты по унифицированной оценке релевантности.

Этап 3. Гибридный поиск пар-кандидатов (Retrieval).

На данном этапе для каждого чанка выполняется поиск семантически близких чанков из других файлов. Доступно три стратегии поиска:

- **broad**, основанная на векторном поиске. Для каждого чанка текущего файла производится поиск k ближайших соседей из других файлов.
- **strict**. Она основана на поиске прямых упоминаний сущностей в текстах заметок. Поиск производится по лемматизированному тексту, что решает проблему словоизменения, несовпадения регистров и использования специальных символов, таких как дефис. (например, имя «Нейронная сеть» будет найдено в тексте «нейронной сети»)
- **combined** объединяет обе стратегии с дедупликацией. Результаты **broad** и **strict** проходят соответствующий им пайплайн извлечения и обработки, после чего фрагменты объединяются и дедуплицируются с помощью коэффициента Жаккара, вычисляемого по леммам. Дедупликация заключается в сравнении содержимого чанков, извлеченных каждой стратегией, друг с другом. Если фрагмент, извлеченный **strict**, будет иметь коэффициент с фрагментом, извлеченным **broad**, больше 0.7 (значение выбрано эмпирически на основе валидационных данных), то чанк от **broad** будет отброшен. Результаты **strict** приоритизируются, так как они считаются более надежными.

Особенности работы и недостатки каждой стратегии описаны в разделе 1.2. Результатом поиска является INITIAL_RETRIEVAL_K фрагментов текста.

Этап 4. Переранжирование кандидатов (Cross-Encoder).

Данный этап проходят только кандидаты, извлеченные векторным поиском, поскольку среди них зачастую много ложноположительных.

Для снижения шума применяется дополнительная фильтрация кандидатов с помощью Cross Encoder. Каждая пара (чанк источника, чанк цели) подается на вход Cross Encoder (модель типа BAAI/bge-reranker-v2-m3 / MiniLM) с инструкцией-вопросом. Cross Encoder вычисляет более точную оценку семантической связности. Пары сортируются по убыванию оценки. В финальный список попадают RERANKER_TOP_K пар, у каждой из которых оценка не ниже порогового значения.

Все прошедшие фильтрацию чанки группируются по файлу-цели. На их основе создаются структуры CandidatePair, содержащие исходный и целевые чанки, а также пути к соответствующим файлам и оценку близости из LanceDB и Cross Encoder.

Результатом этапа является список структур CandidatePair, к которым добавлены результаты лексического поиска, если он был использован. Важно уточнить, что результаты стратегии strict являются более приоритетными, а результаты стратегии broad служат для дополнения выборки и добавляются в итоговый список по остаточному принципу.

Этап 5. Типизация связи (LLM).

Прошедшие фильтрацию пары-кандидаты передаются в языковую модель для определения точного типа семантической связи. Пары группируются по комбинации (файл-источник — файл-цель). Для каждой такой группы отбираются не более LLM_TOP_K_STX наиболее релевантных чанков для формирования контекста на основе оценки Cross Encoder.

Логика запросов к LLM поддерживает параллелизм и батчинг. Количество одновременных запросов к модели ограничено заданным числом и контролируется семафором. Кроме того, в рамках одного запроса может проводиться типизация сразу нескольких ссылок. Это значительно снижает накладные расходы и время обработки.

Важно отметить обработку ошибок: у каждого батча есть три попытки, при израсходовании которых батч откладывается и логируется ошибка. Он будет повторно обработан при следующем запуске.

В рамках запроса модель получает системный промпт со списком допустимых типов связей и инструкциями, а также пользовательский промпт с парами файлов и их контекстом.

Ответ модели возвращается в формате JSON. Его парсинг производится посредством `JsonOutputParser` из `LangChain`, позволяющий восстанавливать синтаксически некорректный JSON. При возврате моделью раздела с размышлениями, он логируется и вырезается для упрощения парсинга.

Для работы с LLM используется `LangChain`, который позволяет интегрировать как локальные модели (например, запущенные через `llama.cpp` [28]), так и облачные провайдеры (OpenAI, Google, и другие).

После каждого батча предсказания модели сохраняются в файл, формируя контрольную точку, что позволяет возобновить типизацию в случае ошибки или аварийного завершения программы. Данная оптимизация призвана экономить вычислительные ресурсы, убирая необходимость повторять выполненную работу.

Этап 6. Сохранение связей (ExportService).

На заключительном этапе типизированные связи сохраняются в одном из трех режимов:

- `inplace`. Связи добавляются непосредственно в исходные Markdown-файлы под специальным заголовком `LINK_HEADER`. При наличии связей

с предыдущих запусков новые связи неdestructивно добавятся к уже имеющимся, не создавая дубликатов.

- `json`. Результаты экспортируются в JSON-файл без изменения исходных заметок. Данный режим создан для удобного переиспользования результатов, например для вычисления метрик.
- `export`. Создается полная копия хранилища с вписанными связями, исходные файлы остаются нетронутыми. Этот формат сохранения полезен для экспериментов, таких как оценка GraphRAG, требующих полноценного хранилища для работы.

Формат ссылки конфигурируется (по умолчанию используется синтаксис `Dataview: - relation:: [[target]]`).

Управление состоянием и отказоустойчивость (MetadataManager).

При первом запуске конвейера стандартные параметры из `config.py` сохраняются в `config.json` внутри директории `.kg_builder/`. Последующие запуски считывают этот файл и выставляют настройки, заданные в нем, что обеспечивает воспроизводимость и повышает удобство использования на разных хранилищах с индивидуальными параметрами.

Опция `--ignore-local-config` заставляет пайплайн проигнорировать `config.json` и использовать значения из `config.py`.

Алгоритм сохраняет свое состояние с помощью создания снимка, который включает текущий этап (сканирование хранилища, сбор кандидатов, типизация, и т.д.), его прогресс и прочие метаданные, необходимые для возобновления работы.

Наибольшей детализацией обладают этапы ранжирования кандидатов и LLM-классификации отношений. У них есть логика контрольных точек, при которых сохраняется список уже обработанных файлов или пар-кандидатов. При возобновлении работы пайплайна на одном из этих шагов в алгоритм будут переданы только необработанные данные.

При последующих запусках KnowledgeGraphBuilder проверяет последнее сохраненное состояние. Если предыдущий запуск был прерван, то он продолжит обработку с того же шага, минуя уже выполненные.

Это сохраняет значительное количество вычислительных ресурсов и времени, особенно на дорогостоящем LLM-этапе.

Все метаданные сохраняются в отдельной директории `.kg_builder/.out/`:

- `candidates.json` — начальные кандидаты до переранжирования;
- `reranked_candidates.json` — кандидаты после переранжирования, обогащенные результатами LLM (поля `llm_type`, `llm_reasoning` дописываются после классификации)
- `predictions_partial.json` — предсказания LLM, обновляемые инкрементально
- `run_state.json` — текущее состояние алгоритма

Специальный флаг `--fresh-start` позволяет перезаписать метаданные, в том числе конфиг и список обработанных файлов. Он может быть полезен при разработке, случаях, когда модифицируется структура метаданных, или они были повреждены.

2.2 Алгоритм GraphRAG

В качестве референсного решения был выбран ObsidianRAG [29]. Его алгоритм состоит из нескольких шагов:

1. Индексация: содержимое markdown-файлов разбивается на чанки размером 1500 символов с перекрытием в 300 символов. Полученные фрагменты вместе с извлеченными из текста ссылками записываются в векторную базу данных ChromaDB, которая создает для них плотные векторные эмбединги.
2. Первоначальный поиск: запрос пользователя обрабатывается с помощью EnsembleRetriever, использующего BM25 и ANN с весами 0.4 и 0.6 соответственно. Извлекается N наиболее релевантных заметок.

3. Переранжирование: полученные файлы передаются в Cross Encoder, например, BAAI/bge-reranker-v2-m3 [30]. В результате фильтрации остаются 6 наиболее близких фрагментов.
4. Расширение контекста: результат дополняется пятью файлами, связанными ссылками с оставшимися.
5. Генерация ответа: собранное содержимое заметок передается в LLM.

У данного пайплайна есть ряд недостатков:

1. Одноэтапное расширение контекста соседями. При таком подходе теряются сложные связи, требующие многошаговых рассуждений. Также возрастает вероятность остаться в семантической ловушке: ситуации, когда новая информация не может быть извлечена из-за сильной разницы в эмбедингах и ключевых словах. Суть ссылок заключается в решении этой проблемы, но автор ими пренебрегает.
2. Сбор контекста без учета его релевантности. В проекте уже используется Cross Encoder для первоначального поиска, его можно легко переиспользовать для фильтрации фрагментов, не относящихся к запросу.

На основе этого решения был разработан улучшенный четырехэтапный пайплайн, поддерживающий типизированные ссылки и многошаговое расширение контекста. Он нацелен на решение фундаментальных проблем эталонного решения. Алгоритм реализован на языке Python (исходный код доступен в папке `src/graphrag`).

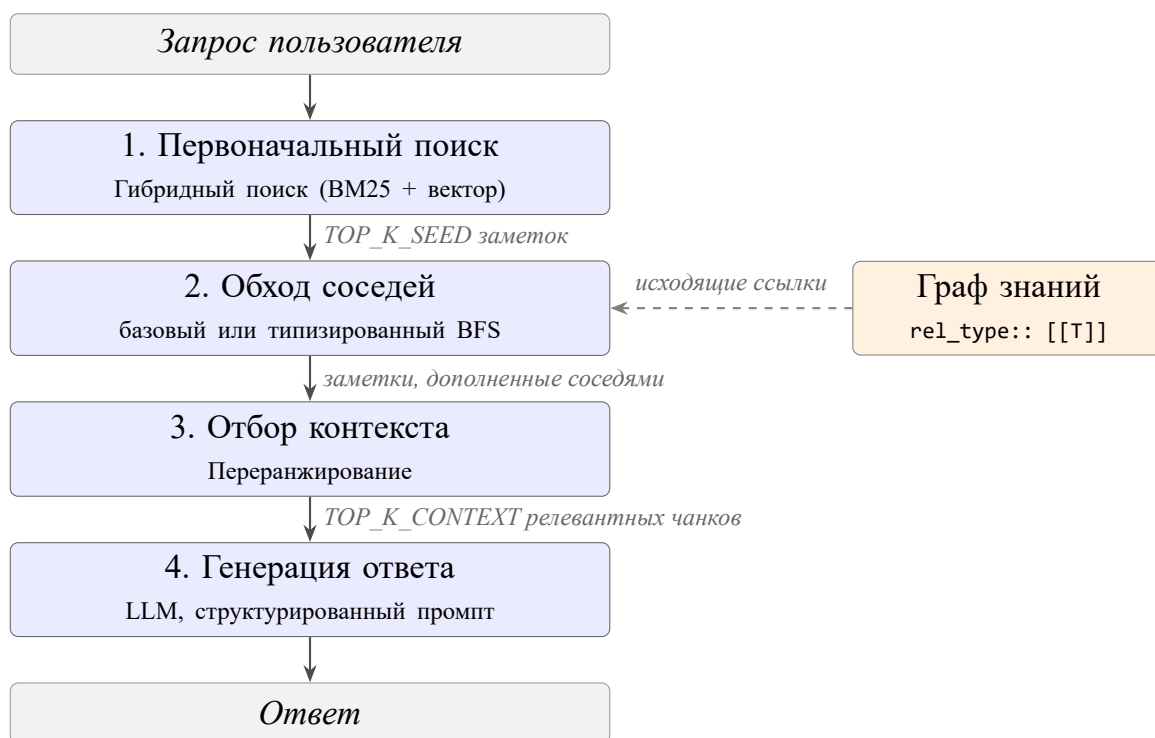


Рис. 3 – Общая схема конвейера GraphRAG

Этап 1. Первоначальный поиск.

На этом этапе выполняется гибридный поиск по LanceDB. Он включает в себя ANN-поиск по плотным эмбедингам и BM25-поиск по лемматизированному тексту. На его основе собирается *TOP_K_SEED* (по умолчанию = 5) наиболее близких фрагментов-кандидатов. Файлы, соответствующие этим чанкам, добавляются в множество посещенных.

В отличие от референсного решения, на данном этапе не выполняется переранжирование заметок с помощью Cross Encoder. Оно было перенесено на шаг 3. Это решение было принято исходя из точности алгоритма извлечения, а также для смягчения критериев для отбора заметок, что потенциально может увеличить полноту собранного контекста.

Этап 2. Обход соседей.

После сбора первоначального множества релевантных заметок, начинается обход их соседей. Изначальное множество формирует очередь обхода, который может быть проведен двумя способами:

- Базовый обход в ширину. Для каждой заметки из очереди формируется множество файлов, на которые ссылается текущая заметка. Все непосещенные добавляются в конец очереди. Типы ссылок при этом игнорируются, как и релевантность содержимого файлов.
- Типизированный обход в ширину. Для каждой заметки, связанной с текущей, формируется запрос в Cross Encoder формата:

`(user_query, relation: target.summary),`

где `user_query` — запрос пользователя `relation` — тип связи, `target` — имя целевой заметки, `summary` — ее краткое содержание (первый абзац). Все кандидаты с оценкой ниже пороговой не добавляются в очередь обхода. Это позволяет фильтровать заметки на этапе сбора контекста, что может снизить количество шума, но также уменьшить полноту контекста из-за раннего отброса.

Обход продолжается до достижения либо максимального числа посещенных заметок (`MAX_VISITED`), либо обхода всех заметок до заданной глубины (`DEFAULT_MAX_HOPS`). Результатом этапа является множество потенциально релевантных заметок.

Этап 3. Отбор содержимого.

Содержимое каждой заметки, отобранной на предыдущем этапе, разбивается на фрагменты с помощью того же алгоритма фрагментации, что используется при индексации хранилища (раздел 2.1).

Для каждой собранной на этапе обхода заметки проводится анализ ее содержимого на соответствие запросу пользователя. Для этого все фрагменты заметок передаются в Cross Encoder вместе с запросом, после чего объединяются с фрагментами, найденными на этапе 1. Из всех чанков сохраняется *TOP_K_CONTEXT* (по умолчанию 10) наиболее релевантных (с наибольшей оценкой), которые сформируют итоговый контекст и будут отправлены в LLM.

Этап 4. Генерация ответа.

Запрос в LLM формируется на основе промпта, который дает модели необходимые инструкции (например, отвечать на языке пользователя, несмотря на язык предоставленного контекста, и опираться только на предоставленные данные), сообщения пользователя и отобранных фрагментов текстов markdown-файлов, разграниченные специальным разделителем.

Предсказание модели ожидается в формате JSON. Они преобразуются в классы с двумя атрибутами: `reasoning` — рассуждения модели и `answer` — финальный ответ, возвращаемый пользователю. В случае неудачного парсинга сырой ответ модели записывается в поле `answer`, `reasoning` остается пустым.

СБОР И ПОДГОТОВКА ДАННЫХ

Для оценки качества работы пайплайна и возможности сравнения его с аналогами необходимо создать репрезентативный датасет, который бы удовлетворял требованиям, описанным в разделе 1.4.

3.1 Поиск данных

В качестве первичной гипотезы рассматривалось использование существующих открытых пользовательских хранилищ в формате Obsidian с их последующей модификацией и фильтрацией. В ходе поиска было отобрано несколько качественных репозиторий:

- личные заметки общего назначения [31];
- структурированная документация коллектива *Hundred Rabbits* [32];
- корпус кулинарных рецептов *based.cooking* [33];
- заметки по программной инженерии [34].

После их детального анализа выявился один общий критический недостаток: нарушалось правило “заметка = сущность”. Наиболее распространенной причиной его нарушения были объединения текстов на разные тематики в одном файле.

Помимо структурного несоответствия был выявлен ряд дополнительных недостатков:

- Семантическая разреженность данных. Некоторые хранилища состояли в основном из дневниковых записей, заметок или рецептов, между которыми практически не было связей, либо они были неоднозначными.
- Несоответствие объема. Почти все найденные хранилища были либо слишком малы, либо слишком велики. К репозиториям с недостающим объемом пришлось бы искусственно добавлять данные, а из слишком

больших удалять, что потенциально может создать массу висячих ссылок и спровоцировать потерю ценных связей и контекста.

- В некоторых репозиториях отсутствует явно указанная лицензия либо присутствуют фрагменты текстов или кода, потенциально нарушающие авторские права. Это делает обработку этих данных рискованной.

Выбор Википедии как основного источника данных.

В качестве следующего источника данных было решено рассмотреть Википедию. У нее есть ряд важных преимуществ:

- Естественная атомарность статей. Зачастую статьи соответствуют одной сущности, что согласуется с правилом “одна заметка = одна сущность”.
- Наличие ссылок. В статьях изначально есть гиперссылки на другие статьи. Эти ссылки образуют граф, к которому можно получить доступ и использовать его как для сбора данных, так и для первоначальной разметки.
- Качество содержимого. Подавляющая часть популярных статей написана в энциклопедическом стиле, обладает высокой плотностью информации, ее последовательным и связным изложением.

Все вышеперечисленное делает Википедию подходящим источником данных для валидационного датасета.

3.2 Создание валидационного датасета

Формирование базового датасета проводилось при помощи Wikipedia API, обращение к которому производилось посредством библиотеки `wikipediaapi` [35]. Данный инструмент позволяет сохранять тексты статей, а также предоставляет список связанных статей

Важно отметить, что статьи имеют специальный раздел, содержащий краткую сводку (`summary`). Он будет использован в качестве контекста пайплайном GraphRAG для фильтрации заметок на этапе сбора контекста.

Формирование структуры элементов датасета.

Поскольку ресурсы ограничены, необходимо определить несколько наиболее показательных сценариев для оценки алгоритма. На основе анализа особенностей работы GraphRAG, описанных в разделе 1.2, был составлен список наиболее показательных структур элементов валидационного датасета и их содержимого:

- **Hub-n-spoke (Hub).** Структура в форме графа-звезды, состоящая из статьи-хаба и ее соседей. Главная задача алгоритма в этом случае — идентификация хаба. Алгоритмы GraphRAG зачастую пытаются обнаружить именно такие узлы, так как они дают непосредственный доступ к большому количеству информации. Тематика датасета — производство полупроводников и микроэлектроника. Итоговый набор статей включает как общие концепции (закон Мура, фотолитография, кремний), так и конкретные технологии или компании (ASML, TSMC, транзисторы, архитектура ARM, Apple M1).
- **Versus.** Структура, состоящая из двух разных тематик, которые местами могут показаться схожими. Например, Apple как компания и apple как яблоко. Такой подход заставит алгоритм работать с содержимым заметок, а не только с названиями сущностей. В собранном наборе данных эта структура реализована на примере явления омонимии: корпорация Apple (включая связанные узлы, например, Стив Джобс, iPhone, macOS) противопоставляется яблоку как фрукту (яблочный пирог, сорт Грэнни Смит) и связанным с ним историческим фактам (Исаак Ньютон).
- **Realistic.** Данный элемент будет содержать две схожие тематики. Его задача — проверка созданных алгоритмом связей на шум. Для датасета было решено связать концепции искусственного интеллекта (перцептроны, машинное обучение, обратное распространение ошибки и т.д.) с их историей.

- **Sequence.** Образец, состоящий из последовательных или вытекающих друг из друга событий или терминов. В качестве предметной области для этого набора была выбрана хронология развития представлений о строении атома (от открытия катодных лучей и электрона до пудинговой модели, экспериментов Резерфорда и модели Бора). Этот подкорпус будет нацелен на проверку способности алгоритма находить сложные, многошаговые связи.
- **Personal.** Элемент, созданный вручную по аналогии с ежедневными заметками и содержащий различные события, планы, наброски идей. В нем смоделированы типичные личные записи: привязки к датам, повседневные задачи (починка велосипеда, покупки), встречи (с научным руководителем, знакомыми) и проекты (работа над дипломом, исследование малых LLM). Данный образец поможет оценить работу в условиях малого и специфичного контекста, отличного от текстов статей.

Визуализация графов знаний, созданных на основе этих подкорпусов, изображена на рисунке А.1.

3.2.1. Алгоритм генерации связей

Для формирования датасета был написан скрипт для выгрузки и обработки статей с использованием `wikipediaapi`, поддерживающий два режима сохранения:

- выгрузка заданного набора статей
- обход соседей заданной статьи в ширину с контролем глубины обхода

Его код расположен в директории `data_parser`.

Во второй стратегии сохранения соседи статьи ранжируются с помощью Cross Encoder модели, в которую передаются пары из кратких сводок соседа и корневой статьи. Отбирается N наиболее релевантных статей с оценкой не ниже порога `cross_encoder_threshold`. Такой подход предотвращает уход алгоритма

от тематики корневой статьи. Далее алгоритм увеличивает глубину рекурсии и аналогично обрабатывает каждого отобранного соседа до тех пор, пока либо не сохранит заданное число статей, либо не обойдет все статьи до достижения заданной глубины рекурсии.

Каждая статья, прошедшая проверку Cross Encoder, подвергается обработке. В нее входит:

- Нормализация названий файлов, включающая замену пробелов на подчеркивания и удаление некоторых служебных символов для однозначного соответствия названий статей названиям файлов.
- Очистка текста от сносок и маркеров (например, [1]), а также удаление служебных секций статьи, которые не несут содержательной информации для GraphRAG (например, “См. также”, “Литература”, “Ссылки”).
- Формирование специального блока ссылок, извлеченных из графа Википедии, на другие статьи. В данном блоке остаются ссылки только на сохраненные статьи, что решает проблему висячих ссылок.

После сохранения статей начинается этап их ручной доразметки. Этот шаг включает:

- Анализ и фильтрация ссылок между сохраненными статьями. Все ссылки, для идентификации которых нет контекста в содержимом статьи, удаляются. Например, в статье «Билл Гейтс» сказано, что он соосновал Microsoft, следовательно связь Билл Гейтс -> (соосновал) Microsoft должна быть оставлена.
- Типизация ссылок. Для каждого корпуса создается онтология типов связей. Для каждого отношения назначается один допустимый тип. Критично обеспечить оптимальный уровень абстракции, чтобы не перегрузить языковую модель: большое количество узких по смыслу ссылок может

показаться лучшим решением, но на практике модель начнет выбирать неверные типы, например из-за недостатка контекста.

Для упрощения разметки и алгоритмов все типы отношений фиксируются на английском языке (например, *Uses*, *Part of*, *Mentions*), несмотря на то, что содержимое некоторых заметок на русском языке.

В каждом подкорпусе присутствует файл *.meta*, в котором содержится список статей с Википедии, на основе которых он был создан, а также список допустимых типов отношений.

Результатом ручной доработки является блок типизированных ссылок в каждой статье, оформленный в синтаксисе плагина *Dataview*:

Related Connections:

- *Mentions::* *[[ASML_Holding]]*
- *Manufactures::* *[[Integrated_circuit]]*
- *Sustains::* *[[Moores_law]]*
- *Is a::* *[[Photolithography]]*

Подкорпус	# заметок	# ссылок	# типов связей	Ср. исх. ссылок на заметку
hub_n_spoke	15	77	12	5.13
realistic	15	79	12	5.27
versus	15	44	14	2.93
sequence	7	23	7	3.29
personal	11	23	7	2.09
Итого	63	246	—	3.90

Таблица 1 – Сводная статистика по эталонным подкорпусам

В таблице 1 приведена информация о собранных данных. *hub_n_spoke* и *realistic* обладают самым высоким средним числом связей на заметку (в среднем 5.2 исходящих ссылки), то есть в них много семантически близких и связанных сущностей. Они помогают оценить пайплайн построения графа на устойчивость к шуму. *versus*, несмотря на идентичное число заметок, имеет вдвое меньше связей, поскольку в нем часть сущностей не имеют никаких

семантических отношений. Аналогичная ситуация в *sequence*: в нем изложена последовательность, в которой каждая сущность в основном связана только с предыдущей и следующей.

Датасеты хранятся в директории `data/test_vaults/gold`.

3.2.2. Алгоритм извлечения контекста

Для оценки *retrieval*-этапа был создан отдельный датасет, состоящий из фрагментов текстов, в которых явно или нет упоминаются сущности и подкорпуса.

В качестве минимального фрагмента текста было решено взять предложение, содержащее упоминание. Это позволит сохранить контекст, а также проверять алгоритм на точность извлечения, то есть долю других предложений, не содержащих упоминание.

Для каждого элемента датасета было проведено ручное извлечение всех фрагментов с упоминаниями. На их основе создан набор JSON-файлов (по одному на элемент эталонного набора данных для оценки алгоритма генерации ссылок), хранящийся в директории `data/test_retrieval/gold`. Каждый файл содержит массив кортежей $(s, e, sent, is_direct)$, где s — файл-источник, e — целевая сущность, $sent$ — предложение, в котором упомянута сущность, is_direct — содержит ли предложение прямое упоминание сущности. Допускается, что для одной пары (s, e) может присутствовать несколько записей с разными извлеченными предложениями.

Важно уточнить, что для соблюдения требований к данным, описанным в разделе 1.4, в датасете используются только чанки, содержащие упоминания сущностей, представленных в подкорпусе.

3.2.3. Алгоритм LLM-типизации связей

Для валидации LLM-классификатора отношений используется дополненная версия описанного выше датасета для оценки *retrieval*. В каждую запись в JSON-файлах было добавлено поле `link_type`, соответствующее

истинному типу отношения. Таким образом финальная структура кортежей имеет следующий вид: $(s, e, sent, is_direct, link_type)$.

Значения *link_type* были взяты из блока ссылок датасета для оценки алгоритма генерации связей. Если сущность упомянута в нескольких предложениях, каждое из них получает одинаковое значение *link_type*.

Вывод

Для оценки качества и устойчивости алгоритмов к разным сценариям использования были сформированы датасеты, различающиеся тематикой, плотностью информацией, сложностью и набором сущностей, для оценки как всего пайплайна, так и для конкретных его этапов. Данный подход задуман для упрощения тонкой настройки гиперпараметров и выбора оптимальных стратегий извлечения контекста и типизации связей.

Выбор в пользу Википедии обеспечил выполнение правила “одна заметка = одна сущность” и позволил получить управляемый объем данных с качественным содержимым, достаточным для оценки GraphRAG.

ЭКСПЕРИМЕНТЫ

4.1 Параметры воспроизводимости

Для обеспечения воспроизводимости экспериментов был зафиксирован сид генерации (seed) на значении, равном 42. Также во всех экспериментах с LLM (как при типизации связей, так и при генерации ответов) температура генерации (temperature) равнялась 0.

4.2 Оценка конвейера построения графа знаний

4.2.1. Извлечение кандидатных пар

Извлечение может быть осуществлено тремя способами: strict, broad и combined. Подробное описание этих стратегий приведено в разделе 2.1.

Описание методологии оценивания изложено в разделе 1.4 «Оценка извлечения контекста».

Стратегия Strict.

В таблице 2 приведены результаты работы стратегии Strict на валидационном датасете для алгоритмов извлечения.

Precision	Recall	F ₁	Время, с
1.0	0.376	0.547	206

Таблица 2 – Результат Strict на валидационном датасете

Список извлеченных фрагментов не содержит ложноположительных результатов, что обеспечивает идеальную точность. Однако такой подход ограничен по полноте, так как он не способен извлечь чанки, в которых искомая сущность не упомянута явно. Таким образом, strict подход задает верхнюю границу по точности и нижнюю по полноте. Стоит отметить, что эта стратегия извлекла все прямые упоминания сущностей, что было достигнуто путем лемматизации текстов.

Общее время работы стратегии на всем датасете составило 206 секунд, что составляет около 41 секунды на подкорпус в среднем. Кроме того, лемматизация текста заняла около 194 секунд, а сам поиск — оставшиеся 12 секунд.

Стратегия Broad.

Качество работы данной стратегии зависит от значений параметров, а также используемых моделей, поэтому для достижения наилучшего качества извлечения был проведен ряд экспериментов.

Первоначальные настройки указаны в таблице 3. Их выбор был основан на значениях, используемых в референсных решениях ObsidianRAG [29] и Neo4j Graph Builder [9], а также на структуре валидационного датасета. Поскольку в нем используются markdown-файлы, имеет смысл приоритезировать разбиение текста на основе заголовков. Модели выбраны на основе их метрик и популярности на HuggingFace.

Инференс Embedding и Cross Encoder моделей осуществлялся локально под управлением операционной системы Arch Linux, оснащенной графическим ускорителем AMD Radeon RX 6700 XT с 12 ГБ видеопамяти. Аппаратное ускорение обеспечивалось платформой ROCm (Radeon Open Compute). Все модели использовались в формате bfloat16, что соответствует их оригинальной спецификации и предотвращает деградацию.

Параметр	Значение
Эмбединги и разбиение текста	
Модель эмбедингов	paraphrase-multilingual-MiniLM-L12-v2 [36]
Размер чанка	4000
Перекрытие чанков	200
Разделители	["\n# ", "\n## ", "\n### ", "\n\n", ". ", "? ", "! ", "\n", " ", ""]
Поиск	
Тор-К	15
Вес векторного поиска	0.5
Переранжирование	
Модель	BAAI/bge-reranker-v2-m3 [30]
Тор-К	5
Порог	0.7

Таблица 3 – Изначальные гиперпараметры алгоритма извлечения

Для поиска оптимальной конфигурации был проведен ряд экспериментов, заключающихся в смене размера чанка, моделей эмбедингов и переранжирования. Их результаты представлены в таблице 4.

Поскольку стратегия Strict уже обеспечивает идеальную точность, основной идеей экспериментов над Broad было увеличение полноты при сохранении приемлемой точности. Допустимый порог Precision был сделан равным 0.5. Его достижение будет первостепенной задачей, после чего приоритет сместится на максимизацию полноты при сохранении точности выше порога.

#	CS	CO	E	K	CE	Порог	P	R	F ₁	Время
1	4000	200	MiniLM	15	bge-r	0.7	0.354	0.199	0.255	34
2	3000	200	MiniLM	15	bge-r	0.7	0.395	0.190	0.256	40
3	5000	300	MiniLM	15	bge-r	0.7	0.301	0.186	0.223	27
4	3000	200	MiniLM	15	jina	0.7	0.553	0.208	0.302	9
5	3000	200	MiniLM	15	jina	0.5	0.510	0.407	0.452	9
6	3000	200	MiniLM	30	jina	0.5	0.500	0.453	0.475	20
7	3000	200	bge-e	30	jina	0.5	0.531	0.468	0.498	23
<i>Strict</i>							1.0	0.376	0.547	3.4

Примечание: Используемые модели:

MiniLM — paraphrase-multilingual-MiniLM-L12-v2 [36],

bge-e — BAAI/bge-m3 [37],

bge-r — BAAI/bge-reranker-v2-m3 [30],

jina — jina-reranker-v2-base-multilingual [38].

CS — Chunk Size, CO — Chunk Overlap, E — Embedding Model, CE — Cross Encoder, Порог — минимально допустимая оценка CE, время в минутах

Таблица 4 – Анализ конфигураций стратегии Broad.

Влияние размера чанка.

Эксперименты 1-3 показали, что фрагмент текста не может быть слишком большим, так как это приводит к потере деталей, что негативно сказывается на точности поиска. Оптимальным по качеству вариантом оказался размер в 3000 символов с перекрытием в 200 символом.

Стоит отметить влияние размера чанков на время работы. Уменьшение размера чанка ведет к увеличению числа фрагментов, которые нужно перевести в векторное пространство, из-за чего растет количество пар для сравнения, и следовательно время работы. Поэтому возникает компромисс: увеличить размер чанков, сделав их менее детализированными, но снизив время работы, либо уменьшить размер, увеличив время.

Роль модели переранжирования.

Модель Cross Encoder также оказалась крайне важной. Переход с bge-r на jina увеличил F₁ до 0.302 (эксперимент 4), но потребовал снижения порога с 0.7 до 0.5 для раскрытия потенциала модели (эксперимент 5). Как итог, F₁ вырос до 0.452.

Изначальное значение порога оценки Cross Encoder для фильтрации результатов было завышенным, из-за чего часть истинно положительных кандидатов отбрасывалась. При bge-r порог в 0.7 был оправдан, так как модель имела низкую точность, но использование jina решило эту проблему, повысив значение Precision с 0.395 до 0.553.

Смена модели также значительно сократила время работы алгоритма (с 40 до 9 минут). Это объясняется вдвое меньшим количеством параметров в модели jina (300 миллионов против 600 у bge-r), а также более современной и вычислительно эффективной архитектурой.

Так как первостепенная цель экспериментов достигнута (порог превысил 0.5), последующие опыты будут нацелены на максимизацию полноты при сохранении приемлемой точности.

Количество извлекаемых чанков.

С ростом точности извлечения и значительным сокращением времени работы появился смысл увеличить число извлекаемых кандидатов при гибридном поиске (Тор-К). Снижение его значения не имеет особого смысла, так как это скорее всего сильно снизит полноту, что противоречит цели.

По результатам эксперимента 6 видно, что увеличение Тор-К спровоцировало рост шума, а также времени работы, но увеличило полноту.

Смена модели эмбеддингов.

На замену MiniLM, переводящую тексты в 384-мерное пространство пришла более тяжелая bge-m3, которая работает с 1024-мерными эмбеддингами. Идея была в улучшении сохранения детализации смыслов в текстах. Как итог, рост точности и полноты на 0.031 и 0.015 соответственно по сравнению с результатами эксперимента 6.

Стратегия Combined.

В данной стратегии использовалась оптимальная конфигурация Broad. Результаты представлены в таблице 5. Видно, что Broad вносит весомый вклад: полнота выросла практически вдвое (с 0.376 до 0.685), но ожидаемо упала точность (с 1 до 0.675). Полученные результаты подтверждают теорию: данная стратегия действительно является компромиссом между высокой точностью и полнотой.

Precision	Recall	F_1
0.675	0.685	0.680

Таблица 5 – Результат Combined на валидационном датасете

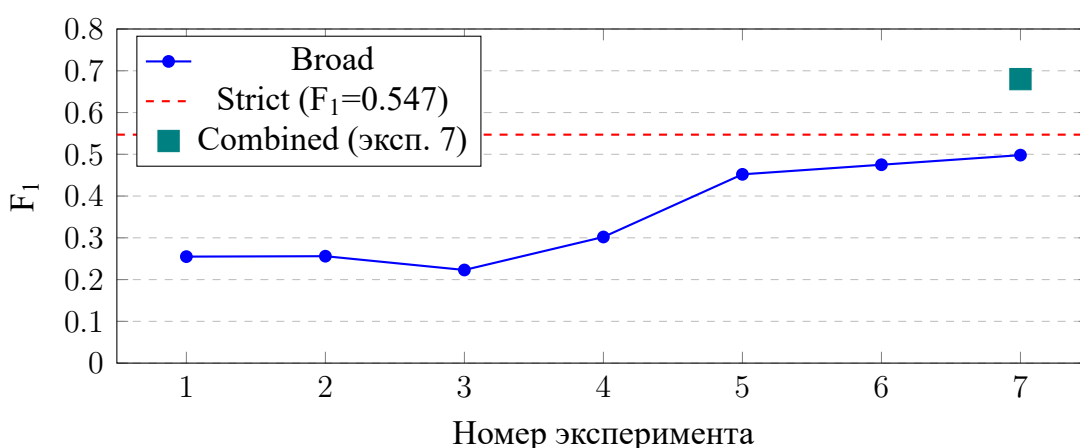


Рис. 4 – Динамика F_1 стратегии Broad по экспериментам. Красная пунктирная линия — уровень Strict-стратегии. Квадрат — результат Combined для лучшей конфигурации.

4.2.2. Типизация связей

В рамках данной серии экспериментов рассматривалась способность больших языковых моделей корректно определять семантический тип связи между сущностями на основе собранных контекстов. Для оценки использовался вручную размеченный валидационный датасет.

Поскольку важна как точность, так и полнота классификации, в качестве основной метрики был выбран F_1 -score. Кроме того, так как в данных

присутствует дисбаланс, уместнее всего использовать Weighted F_1 . Идея экспериментов заключалась в максимизации основной метрики путем поиска оптимального промпта и модели.

Результаты опытов представлены в таблице 6. LLM модели использовались посредством API облачных платформ. Для Qwen-3-235B [39] использовался Cerebras Inference API [40], для Gemma-3-27B [41] и Gemma-4-31B [42] — Google Vertex AI [43]. Оценивание производилось согласно методологии, описанной в разделе 1.4 «Оценка LLM-типизации»

Модель	Промпт	Batch	P	H	R	S	V	W. F_1
Gemma-3-27B	base	1	0.346	0.340	0.351	0.459	0.522	0.393
Gemma-3-27B	base	5	0.446	0.325	0.321	0.451	0.573	0.387
Gemma-3-27B	base	10	0.378	0.428	0.363	0.392	0.572	0.423
Qwen-3-235B	base	1	0.313	0.454	0.272	0.454	0.526	0.392
Gemma-3-27B	few-shot	1	0.372	0.436	0.444	0.341	0.559	0.445
Qwen-3-235B	few-shot	1	0.498	0.390	0.434	0.558	0.482	0.445
Gemma-4-31B	few-shot	1	0.856	0.578	0.627	0.786	0.758	0.669

Примечание: Используемые промпты доступны в репозитории проекта по пути: artifacts/prompts/classification.

Таблица 6 — Weighted F_1 классификации типов связей

Влияние пакетного промптинга.

Увеличение количества пар заметок, передаваемых модели в рамках одного запроса, в среднем снижает среднюю оценку, однако в некоторых случаях она растет. Увеличение может быть вызвано утечкой контекста: модель получает дополнительные данные из других элементов пакета. Это видно в случае Personal, Hub-n-Spoke и Versus, где размер содержимого заметок мал, и есть кластеры из тематически связанных заметок. Тем не менее, важно не перегрузить модель лишним контекстом: при классификации конкретной связи контекст для других считается шумом и не должен быть учтен. Не все модели могут корректно разграничить содержимое, из-за чего начинают путаться и делать ошибочные выводы, ухудшая качество классификации. Как наиболее честный вариант был выбран размер пакета равный 1.

Влияние промпта.

Добавление в промпт примеров и описаний типов связей, которые в них используются увеличило значение Overall F_1 как для Gemma-3-27B с 0.392 до 0.423 (+0.031), так и для Qwen-3-235B с 0.392 до 0.445 (+0.135). Это подтверждает чувствительность моделей к инструкциям и примерам: они помогают им лучше понять специфику задачи и не уйти в сторону при рассуждении. Важно точно определить каждый тип связи для устранения неоднозначностей. Кроме того, результаты показывают, что качество более крупной модели больше зависит от промпта.

Влияние выбора модели.

Эксперименты показали, что эффективность архитектуры важнее числа параметров. Новая Gemma-4-31B (31 млрд параметров) значительно превосходит по качеству Qwen-3-235B (235 млрд параметров), а Gemma-3-27B (27 млрд параметров) оказалась в ней наравне.

Анализ ошибок классификации.

Несмотря на рост общего значения Weighted F_1 , имеется несколько общих ошибок:

- Злоупотребление универсальными типами. Тип *mentions* является самым популярным в датасете (40 связей, $F_1 = 0.23\text{--}0.39$). Он играет роль резервного класса: он назначается в случае, когда связь есть, но невозможно однозначно назначить другой класс. Модель может слишком часто назначать этот тип из-за его универсальности. Эта проблема решается дополнительными инструкциями и примерами в промпте, явно указывающими смысл этой связи.
- Использование близких по смыслу связей. Классы *encompasses*, *contains*, *incorporates* и другие имеют высокое семантическое сходство. Большое их

количество создает путаницу: модель не способна выявить семантической разницы между ними, поэтому с высокой вероятностью может использовать неверный тип. Проблема обостряется в реальных условиях неточного контекста, который создает алгоритм извлечения контекста. Эффективным решением является объединение подобных классов в один более универсальный.

4.3 Сравнение сгенерированных ссылок с эталонными

Перед оценкой GraphRAG на сгенерированных ссылках необходимо их проанализировать. Оценка проводилась в двух режимах: без учета типа (оценивается только факт наличия связи между заметками) и с учетом типа (оценивается как наличие связи, так и правильность предсказанного типа) для анализа качества ссылок для использования в каждой типе обхода. Использовались лучшие конфигурации извлечения (стратегия Combined) и типизации (Gemma-4-31B, few-shot промпт, $batch = 1$).

Подкорпус	Без учета типа (Untyped)			С учетом типа (Typed)		
	P	R	F ₁	P	R	F ₁
hub_n_spoke	0.810	0.883	0.845	0.643	0.701	0.671
personal	0.909	0.435	0.588	0.909	0.435	0.588
realistic	0.771	0.810	0.790	0.687	0.722	0.704
sequence	0.733	0.957	0.830	0.533	0.696	0.604
versus	0.727	0.909	0.808	0.545	0.682	0.606
Взвеш. усред.	0.776	0.829	0.802	0.635	0.679	0.656

Таблица 7 – Оценка качества сгенерированных связей: факт наличия (Untyped) и точное совпадение типа (Typed)

Точность извлеченных ссылок без учета их типа находится на высоком уровне: $F_1 = 0.802$, при этом полнота также имеет близкое к идеальному значение $R = 0.829$, что говорит о способности алгоритма находить практически все ссылки. Наилучшие результаты наблюдаются в подкорпусах *sequence* ($R = 0.957$) и *versus* ($R = 0.909$).

Введение типов ссылок как критерия оценивания снизило значения метрик:

F_1 упал до 0.656. Исключением является образец `personal`, в котором метрики не изменились. Это означает, что алгоритм безошибочно классифицировал все связи. Такой результат был ожидаем, он напрямую коррелирует с результатами раздела 4.2.2.

4.4 Оценка конвейера GraphRAG

В таблице 8 представлены первоначальные параметры алгоритмов. Они выбраны на основе количества валидационных данных и референсного решения. Значения подобраны так, чтобы заставить алгоритмы обходить граф, а не собирать контекст всех заметок разом.

Описание параметра	Переменная	Значение
Максимальная глубина обхода графа	MAX_HOPS	2
Ширина Beam Search	BEAM_WIDTH	4
Порог отсеечения узлов по релевантности	SCORE_THRESHOLD	0.2
Количество первоначальных заметок	TOP_K_SEED	3
Итоговое количество чанков в контексте	TOP_K_CONTEXT	10

Таблица 8 – Значения гиперпараметров алгоритма GraphRAG по умолчанию

Оценка проводилась в соответствии с методологией, описанной в разделе 1.4 «Полная оценка GraphRAG».

4.4.1. Анализ референсных решений

Для оценки разработанных алгоритмов было проведено их комплексное сравнение с ObsidianRAG и Neo4j (Graph Builder и его же интегрированная реализация RAG). В качестве модели для генерации ответов была выбрана Gemma-3-27B, для оценки Gemma-4-31B.

Подкорпус	CR	CP	F	AC
hub_n_spoke	0.802	0.701	0.955	0.542
personal	0.625	0.615	1.000	0.424
realistic	0.886	0.707	0.898	0.505
sequence	1.000	0.824	0.967	0.582
versus	0.617	0.467	0.883	0.428
Среднее	0.786	0.663	0.941	0.496

Таблица 9 – Результаты решения ObsidianRAG

Результаты оценки референсного решения ObsidianRAG приведены в таблице 9. Явно видны недостатки логики расширения контекста: в `personal` и `versus` наблюдается низкий CR (0.625 и 0.617 соответственно). В этих образцах есть два кластера, которые практически друг с другом не связаны. Наивный сбор контекста соседних узлов не позволит получить информацию из другого кластера, а лишь увеличит шум, что видно по низким значениям CP (0.467 в случае `versus` и 0.615 в случае `personal`). В других образцах, где заметки связаны большим числом ссылок, CR не ниже 0.8, а CP не ниже 0.7, что говорит о хорошей работе алгоритма.

Как результат, следует и низкий AC: контекст зашумлен и в нем недостаточно данных для предоставления полного и точного ответа.

Подкорпус	CR	CP	F	AC
hub_n_spoke	1.000	0.782	0.951	0.598
personal	1.000	0.875	0.984	0.612
realistic	0.985	0.842	0.975	0.620
sequence	0.980	0.855	0.971	0.640
versus	0.920	0.751	0.905	0.540
Среднее	0.977	0.821	0.957	0.602

Таблица 10 – Результаты решения Neo4j на сгенерированном ГЗ

В таблице 10 представлены результаты Neo4j, полученные на сгенерированном Neo4j Graph Builder ГЗ.

Решение имеет значительно более высокие значения CR, что обусловлено более продвинутой логикой сбора контекста. Среднее значение CR достигло 0.977, в то время как у ObsidianRAG оно лишь 0.786. Также виден большой

прирост метрики на подкорпусах *personal* (с 0.625 до 1.000) и *versus* (с 0.617 до 0.920).

Кроме того, CR также вырос — в среднем 0.821 против 0.663 у ObsidianRAG. Это говорит об отличной способности Neo4j фильтровать контекст.

Как следствие, увеличились значения AC: среднее значение выросло на 0.106 балла (с 0.496 до 0.602). Прирост особенно заметен на проблемных для ObsidianRAG подкорпусах *personal* (+0.188) и *versus* (+0.112).

Во всех случаях наблюдается высокий Answer Faithfulness: модель не пытается додумать ответ, а использует приведенные данные.

4.4.2. Сравнение собственных алгоритмов с аналогами

Поскольку основным преимуществом алгоритмов, представленных в данной работе, является многошаговый обход, основная цель которого увеличение CR, имеет смысл сфокусироваться только на полноте контекста и качестве ответа при сравнении разработанных алгоритмов с эталонным. Сравнение представлено в таблице 11.

Подкорпус	Данные	Ref		Base		Typed		Neo4j	
		CR	AC	CR	AC	CR	AC	CR	AC
hub	GT	0.802	0.542	1.000	0.587	0.958	0.565	—	—
	GEN	0.750	0.516	0.958	0.535	0.958	0.561	1.000	0.598
personal	GT	0.625	0.424	1.000	0.597	1.000	0.579	—	—
	GEN	0.625	0.429	1.000	0.602	1.000	0.642	1.000	0.612
realistic	GT	0.886	0.505	0.977	0.606	0.977	0.601	—	—
	GEN	0.864	0.522	0.977	0.637	0.977	0.602	0.985	0.620
sequence	GT	1.000	0.582	0.925	0.580	0.975	0.631	—	—
	GEN	0.975	0.555	0.925	0.614	0.925	0.600	0.980	0.640
versus	GT	0.617	0.428	0.900	0.524	0.788	0.507	—	—
	GEN	0.608	0.388	0.900	0.550	0.788	0.520	0.920	0.540
Среднее	GT	0.786	0.496	0.960	0.579	0.940	0.577	—	—
	GEN	0.764	0.482	0.952	0.587	0.930	0.585	0.977	0.602

Таблица 11 – Сравнение собственных алгоритмов с аналогами на размеченном (GT) и сгенерированном (GEN) графах.

Оценка GraphRAG на размеченных данных

Видно, что модифицированная логика обхода действительно увеличила количество собираемой информации по сравнению с ObsidianRAG, позволив получить практически идеальную полноту контекста. Это оказалось особенно заметно в случае `personal` и `versus`, где изначально значение CR было самым низким. В их случае прирост составил 0.375 и 0.283 соответственно в случае Base, 0.375 и 0.171 в случае Typed.

Дополнительный контекст помог языковой модели дать более точный ответ, что также увеличило среднее значение AC с 0.496 до 0.579 (Base) и 0.577 (Typed)

Оценка GraphRAG на сгенерированном графе

Для получения первоначальных данных и оценки качества работы алгоритмов на автоматически построенных ГЗ в следующем эксперименте размеченный ГЗ был заменен на граф, построенным пайплайном генерации связей. Результаты приведены в таблице 11.

Видно, что замена графа минимально влияет на качество работы GraphRAG: значения метрик колеблются в диапазоне сотых долей.

Переход на сгенерированные связи незначительно снизил CR: для базового алгоритма среднее значение упало с 0.960 до 0.952, а для типизированного — с 0.940 до 0.930. Это падение можно объяснить пропуском неочевидных связей конвейером построения ГЗ, из-за чего усложняется навигация по графу.

Интересным наблюдением является рост AC (с 0.579 до 0.587 для Base и с 0.577 до 0.585 для Typed). Особенно заметно увеличение в подкорпусе `personal` для Typed-обхода (с 0.579 до 0.642) и в `realistic` для Base-обхода (с 0.606 до 0.637). Возможно, что сгенерированные связи в некоторых случаях более эффективны для сбора релевантного контекста, так как пайплайн мог иначе типизировать связи или создать новые, которые помогли извлечь нужные данные.

Алгоритм Base показал себя лучше в большинстве случаев как при сборе контекста, так и генерации ответов.

Результаты работы предложенных алгоритмов оказались сопоставимы с Neo4j: в среднем алгоритм Base уступает ему всего на 0.025 по полноте и на 0.015 по точности ответа, алгоритм Typed на 0.047 и 0.017 соответственно. Это доказывает эффективность разработанных алгоритмов.

4.4.3. Влияние промпта и языковой модели

Для исследования влияния различных факторов и компонент GraphRAG на качество ответа LLM была проведена серия экспериментов над решениями, представленными в данной работе. Все замеры проводились на размеченных данных.

Базовый промпт против few-shot.

На основе экспериментов над LLM-классификацией связей был сделан вывод о важности указания точных инструкций и конкретных примеров в промпте. В базовом промпте уже есть основные указания, такие как придерживание предоставленного контекста и ответ на языке запроса. К ним были добавлены конкретные развернутые примеры и требования модели к рассуждениям перед ответом. Модели для оценки и генерации остались теми же.

Алгоритм	Промпт	H	P	R	S	V	AC avg
Base	Базовый	0.587	0.597	0.606	0.580	0.524	0.579
	Few-shot	0.595	0.572	0.604	0.618	0.569	0.592
Typed	Базовый	0.565	0.579	0.601	0.631	0.507	0.577
	Few-shot	0.600	0.611	0.606	0.629	0.555	0.580

Примечание: Используемые промпты доступны в репозитории проекта по пути: `artifacts/prompts/graphrag`.

Таблица 12 – Сравнение базового и few-shot промптов на алгоритмах Base и Typed (метрика AC, модель Gemma-3-27B)

Новый промпт продемонстрировал прирост AC в среднем лишь +0.13. Прирост оказался ниже, чем в предыдущих сравнениях, вероятно, из-за корректно сформулированных инструкций в базовом промпте. Можно сделать вывод, что наличие примеров все же дает небольшой прирост корректности ответа языковой

модели.

Анализ влияния языковой модели.

В этом эксперименте модель для генерации будет заменена на Gemma-4-31B, а моделью-судьей будет Gemini-3-flash [44].

Датасет	Алгоритм	Эксп. 2 (Gemma-3-27B)			Эксп. 3 (Gemma-4-31B)		
		CP	CR	AC	CP	CR	AC
hub	Base	0.749	1.000	0.595	0.660 ↓	0.875 ↓	0.698 ↑
	Typed	0.796	0.958	0.600	0.707 ↓	0.833 ↓	0.709 ↑
personal	Base	0.893	1.000	0.572	0.750 ↓	1.000 =	0.796 ↑
	Typed	0.892	1.000	0.611	0.729 ↓	0.917 ↓	0.770 ↑
realistic	Base	0.848	0.977	0.604	0.426 ↓	1.000 ↑	0.738 ↑
	Typed	0.851	0.977	0.606	0.491 ↓	0.955 ↓	0.792 ↑
sequence	Base	0.879	0.925	0.618	0.434 ↓	0.765 ↓	0.734 ↑
	Typed	0.878	0.975	0.629	0.429 ↓	0.765 ↓	0.706 ↑
versus	Base	0.734	0.900	0.569	0.519 ↓	0.655 ↓	0.623 ↑
	Typed	0.696	0.813	0.555	0.547 ↓	0.568 ↓	0.576 ↑
Среднее	Base	0.821	0.960	0.592	0.558 ↓	0.859 ↓	0.718 ↑
	Typed	0.823	0.945	0.600	0.581 ↓	0.808 ↓	0.711 ↑

Таблица 13 – Сравнение моделей Gemma-3-27B и Gemma-4-31B (few-shot промпт)

Результаты представлены в таблице 13. Прирост AC составил в среднем +0.128, при этом CP упал в среднем на -0.263. Сильнее всего CP снизился на realistic (с 0.848 до 0.426) и sequence (с 0.879 до 0.434). Снижение CP обусловлено более строгой оценкой модели. Однако рост AC показывает, что более мощная модель для генерации ответов способна абстрагироваться от шума и дать более точный ответ.

Сравнение типизированного обхода с обычным.

У типизированного обхода выше CP (среднее: 0.581 против 0.558 у Base), но ниже CR (0.808 против 0.859 у Base). Эти результаты показывают влияние фильтрации заметок при обходе графа: тематически нерелевантные файлы отсекаются, но при этом также теряется возможность сделать дополнительный

шаг через этот файл, чтобы оценить его соседей, из-за чего упускается часть информации.

При этом нельзя сделать однозначный вывод о влиянии учета типов ссылок на корректность ответов. В плотно связанных подкорпусах, таких как *realistic*, типизированный подход обладает большей точностью контекста, но на разреженных графах доминирует обычный обход, видимо, потому что он позволяет делать дополнительные шаги через нерелевантные узлы, что иногда позволяет добраться до необходимой информации.

4.4.4. Анализ влияния графа.

Результаты в таблице 14 уже показали устойчивость и эффективность сгенерированных ссылок. Данный эксперимент нацелен подтвердить, что этот результат сохраняется при использовании более мощных и строгих LLM как для генерации ответа, так и для оценки.

Датасет	Алгоритм	Разметка			Сгенерированные		
		CP	CR	AC	CP	CR	AC
hub	Base	0.660	0.875	0.698	0.689 ↑	0.833 ↓	0.682 ↓
	Typed	0.707	0.833	0.709	0.707 =	0.833 =	0.716 ↑
personal	Base	0.750	1.000	0.796	0.729 ↓	0.958 ↓	0.762 ↓
	Typed	0.729	0.917	0.770	0.729 =	0.917 =	0.770 =
realistic	Base	0.426	1.000	0.738	0.426 =	1.000 =	0.738 =
	Typed	0.491	0.954	0.792	0.476 ↓	0.977 ↑	0.774 ↓
sequence	Base	0.434	0.765	0.734	0.434 =	0.765 =	0.734 =
	Typed	0.429	0.765	0.706	0.429 =	0.765 =	0.706 =
versus	Base	0.519	0.655	0.623	0.519 =	0.655 =	0.623 =
	Typed	0.547	0.568	0.576	0.547 =	0.568 =	0.576 =
Среднее	Base	0.558	0.859	0.718	0.559 ↑	0.842 ↓	0.708 ↓
	Typed	0.581	0.807	0.711	0.578 ↓	0.812 ↑	0.708 ↓

Таблица 14 – Сравнение качества работы GraphRAG между размеченным графом и сгенерированным.

На основе полученных результатов (таблица 14) можно сделать вывод, что использование сгенерированных связей практически не влияет на качество работы GraphRAG.

Падение метрики AC при переходе на автоматически построенный граф в среднем не превышает 0.01 (с 0.718 до 0.708 для Base-обхода и с 0.711 до 0.708 для Typed-обхода). Самое большое падение AC наблюдается в случае базового алгоритма в образце `personal` и составляет 0.034 (с 0.796 до 0.762).

Колебание значений Context Precision и Context Recall незначительны, что говорит о сопоставимом качестве связей с ручной разметкой. Эти результаты говорят об высокой устойчивости к шуму алгоритма построения ГЗ.

4.5 Выводы и ограничения исследования

Опыты над алгоритмом конструирования графа определили оптимальную конфигурацию извлечения контекста: переход к рекурсивному алгоритму вместе с использованием Jina как модели переранжирования позволил достичь $F_1 = 0.498$, что в комбинации с результатами стратегии Strict дает оптимальный баланс между точностью и полнотой извлеченных фрагментов.

Эксперименты над LLM-классификацией связи доказали критичность использования few-shot промпта с четкими инструкциями, а также важность современной архитектуры языковой модели: Gemma-4-31B показала результат Overall $F_1 = 0.669$, превзойдя более крупную Qwen-3-235B ($F_1 = 0.445$). Также важным оказалось объединение семантически схожих классов ссылок для упрощения классификации языковой моделью.

Оценка предложенных алгоритмов GraphRAG через RAGAS показала их эффективность: по результатам видно явное превосходство над ObsidianRAG и сопоставимое с Neo4j качество работы. Разработанная логика сбора контекста повысила CR по всех подкорпусах вплоть до 1.0. Сравнение типизированного обхода с обычным показало гибкость системы: алгоритм может точно извлекать достаточный объем информации в разных сценариях. Типизированный обход лучше всего подходит для работы с шумными, плотно связанными графами. Базовый, в свою очередь, демонстрирует отличные результаты на разреженных графах, требующих многошаговое извлечение контекста.

Финальное сравнение работы предложенных алгоритмов GraphRAG на

сгенерированных ссылок показало высокое качество пайплайна для построения графов знаний в Obsidian. Конвейер устойчив к шуму и может справляться с неоднозначными и сложными ситуациями.

Важно отметить, что во всех экспериментах значения метрики Answer Faithfulness были не ниже 0.88, что говорит об отсутствии галлюцинаций и использовании языковыми моделями приведенного контекста.

Несмотря на все достижения, у проведенного исследования есть ряд ограничений:

- Размер и однотипность тестовых данных. Датасет содержит всего 5 подкорпусов, которые вместе насчитывают 63 заметки и 246 ссылок (таблица 1). Их содержимое в основном – статьи Википедии. Это сильно ограничивает значимость метрик и делает невозможным оценку масштабируемости предложенных подходов и эксперименты с гиперпараметрами GraphRAG.
- Ограниченность онтологии. Набор используемых данных содержит всего несколько схожих тематических групп. Исходя из этого, невозможно предположить поведение системы на данных из другой области.
- Зависимость от качества разметки. Валидационных датасет создавался одним человеком, что вносит долю субъективности в типы некоторых связей.

ЗАКЛЮЧЕНИЕ

По результатам проведенного исследования можно сформулировать следующие выводы:

Во-первых, на текущий момент на рынке отсутствует оптимальное решение для автоматизированного создания ГЗ в Obsidian. Представленные инструменты имеют компромиссы. Существующие модули, работающие поверх корпоративных графовых баз данных (такие как Neo4j Graph Builder и MemGraph Unstructured2Graph), требуют дополнительных ресурсов для работы, чрезмерно зависят от LLM и не поддерживают инкрементальные обновления. Облачные сервисы для ведения заметок имеют риски утечки данных и привязывают пользователей к экосистеме. Эти недостатки подтверждают необходимость создания легковесного и независимого от поставщиков алгоритма, сохраняющего приватность пользовательских данных и оптимизированного под высокодинамичные хранилища.

Во-вторых, анализ готовых решений позволил спроектировать и реализовать эффективный, отказоустойчивый конвейер для автоматизированного построения ГЗ в Obsidian. Разработанное решение совмещает гибридный поиск (BM25 и семантический по плотным эмбедингам BGE-M3 на базе LanceDB) с фильтрацией через Cross Encoder (Jina) и типизацию связей с помощью LLM (Gemma-4-31B). Алгоритм также имеет модуль для сохранения состояния конвейера (MetadataManager), позволяющий возобновлять работу алгоритма с того же этапа, и модуль для состояния хранилища (VaultManager), проверяющий заметки на наличие изменений посредством сравнения снимка состояния файловой системы с предыдущего запуска и вычисления SHA-256 хеша их содержимого, что обеспечивает инкрементальные обновления. Такой подход позволяет экономить вычислительные ресурсы, что крайне важно для индивидуального использования.

В-третьих, для тестирования собственного решения, в частности для

сравнения типизированного GraphRAG с нетипизированным, был создан модифицированный алгоритм GraphRAG, основанный на решении ObsidianRAG. Его особенностями стали учет типов связей при сборе контекста и многошаговый обход.

В-четвертых, для оценки системы и сравнения с аналогами была разработана методология тестирования и собран валидационный набор данных. Выбор Википедии как основного источника данных позволил соблюсти строгие ограничения содержимого заметок и структуры хранилища. Собранный датасет состоит из 63 заметок и 246 связей. Он разделен на 5 сценариев, каждый из которых проверяет работоспособность алгоритма построения ГЗ в одном из сложных и показательных случаев, таких как омонимия, пересекающиеся темы, многошаговая хронология и недостаток контекста.

В-пятых, был проведен ряд сравнений и экспериментов по улучшению полученного решения. Полученные результаты подтвердили высокую эффективность и точность работы предложенных алгоритмов. Выявлено, что оптимальной стратегией извлечения контекста является гибридный поиск: она обеспечивает наилучший баланс точности и полноты, достигая значения F1-меры 0.680. Для типизации связей лучше всего использовать LLM с современной архитектурой вместе с few-shot промптом, детально описывающим допустимые типы и задачу. Такой подход позволил получить значение взвешенной F1-меры в 0.669 (и 0.802 без учета типизации). Оценка собственного алгоритма GraphRAG доказала целесообразность предложенных улучшений, продемонстрировав значительный рост метрики полноты извлеченного контекста (Context Recall, CR), достигшей 0.96 в среднем и 1.0 на подкорпусах hub и personal в случае нетипизированного обхода и 0.940 в среднем для типизированного. При этом алгоритм показал высокую устойчивость к галлюцинациям: метрика достоверности ответа (Answer Faithfulness) стабильно превышала 0.88. Для сравнения, ObsidianRAG имеет среднее значения CR равное 0.786 и точность ответа (Answer Correctness, AC) на уровне 0.496. Важным достижением стало

минимальное падение качество при переходе от созданного вручную ГЗ к сгенерированному: снижение CR составило 0.013, а AC — 0.01 в среднем по обоим алгоритмам. Также было проведено сравнение качества извлечения со сгенерированным с помощью Neo4j Graph Builder ГЗ. Данное решение ожидаемо продемонстрировало наивысшие значения метрик: $CR = 0.977$ и $AC = 0.602$.

Тем не менее, разработанный алгоритм является практически применимым решением, предлагающим оптимальный компромисс и устраняющим когнитивное «узкое горлышко», заключающееся в ручном связывании многочисленных заметок в системах управления знаниями. Несмотря на отставание от корпоративного решения по метрике AC, предложенная система обеспечивает сопоставимую полноту контекста, а также поддерживает инкрементальные обновления и не зависит от тяжеловесных LLM. Конвейер позволяет без участия человека превратить хранилище в граф знаний, который можно использовать для эффективного извлечения данных с помощью GraphRAG.

Несмотря на достижение отличных результатов, в работе есть ряд ограничений. Прежде всего это небольшой объем датасета (63 заметки), однотипность его содержания и узкая онтология типов связей. Также недостатком можно считать отсутствие дополнительных шагов, увеличивающих эффективность GraphRAG, таких как создание сообществ.

Дальнейшее развитие решения заключается в проведении оценки на хранилищах большего размера, состоящих из сотен или тысяч реальных обезличенных заметок различных форматов и тематик. Кроме того, перспективно добавить генерацию сообществ и автоматическую адаптацию используемых типов связей (объединение, добавление и удаление типов). Развитие предложенной системы позволит создать полностью автономный алгоритм конструирования и обновления ГЗ в Obsidian.

Исходный код разработанных алгоритмов и валидационные наборы данных размещены в репозитории на платформе GitHub [45].

СПИСОК ЛИТЕРАТУРЫ

На английском языке

- [1] *Ahrens S.* How to Take Smart Notes: One Simple Technique to Boost Writing, Learning and Thinking. — CreateSpace Independent Publishing Platform, 2017.
- [2] *Dynalist Inc.* Obsidian: A knowledge base that works on local Markdown files [Электронный ресурс]. — Режим доступа: [https : / / obsidian . md/](https://obsidian.md/) (дата обращения: 09.05.2026).
- [3] *Forte T.* Building a second brain: A proven method to organize your digital life and unlock your creative potential. — 2022.
- [4] Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks / P. Lewis [и др.] // *Advances in Neural Information Processing Systems*. Т. 33. — 2020. — С. 9459—9474.
- [5] *Robertson S., Zaragoza H.* The Probabilistic Relevance Framework: BM25 and Beyond // *Foundations and Trends in Information Retrieval*. — 2009. — Т. 3, № 4. — С. 333—389.
- [6] Dense Passage Retrieval for Open-Domain Question Answering / V. Karpukhin [и др.] // *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. — 2020. — С. 6769—6781.
- [7] Retrieval-Augmented Generation for Large Language Models: A Survey / Y. Gao [и др.] // *arXiv preprint arXiv:2312.10997*. — 2023.
- [8] From Local to Global: A Graph RAG Approach to Query-Focused Summarization / D. Edge [и др.] // *arXiv preprint arXiv:2404.16130*. — 2024.
- [9] *Neo4j Labs.* LLM Graph Builder [Электронный ресурс]. — 2026. — Режим доступа: <https://github.com/neo4j-labs/llm-graph-builder> (дата обращения: 06.05.2026).

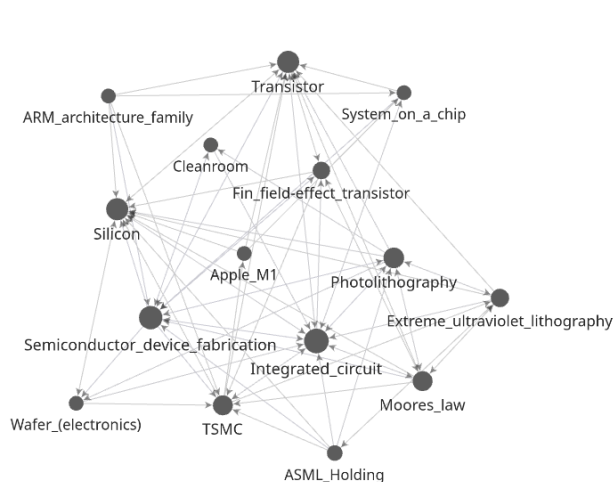
- [10] *Memgraph*. Unstructured2Graph [Электронный ресурс]. — 2026. — Режим доступа: <https://github.com/memgraph/ai-toolkit/tree/main/unstructured2graph> (дата обращения: 06.05.2026).
- [11] *Mem Labs*. Mem.ai: The self-organizing workspace. — Режим доступа: <https://mem.ai/> (дата обращения: 08.05.2026).
- [12] A Survey of Large Language Models / W. X. Zhao [и др.] // arXiv preprint. — 2023. — arXiv: 2303.18223.
- [13] Survey of hallucination in natural language generation / Z. Ji [и др.] // ACM Computing Surveys. — 2023. — Т. 55, № 12. — С. 1—38.
- [14] *Nogueira R., Cho K.* Passage Re-ranking with BERT // arXiv preprint. — 2019. — arXiv: 1901.04085.
- [15] *Brenan M.* Dataview: A data index and query language over your personal knowledge base [Электронный ресурс]. — 2024. — Режим доступа: <https://github.com/blacksmithgu/obsidian-dataview> (дата обращения: 10.05.2026).
- [16] Unifying Large Language Models and Knowledge Graphs: A Roadmap / S. Pan [и др.] // IEEE Transactions on Knowledge and Data Engineering. — 2024. — Т. 36, № 7. — С. 3007—3026.
- [17] *Chase H.* LangChain. — Режим доступа: <https://github.com/langchain-ai/langchain> (дата обращения: 25.02.2026).
- [18] An optimized pipeline for automatic educational knowledge graph construction [Электронный ресурс] / Q. U. Ain [и др.] // arXiv preprint arXiv:2509.05392. — 2025. — DOI: 10.48550/arXiv.2509.05392.
- [19] SqueezeBERT: What can computer vision teach NLP about efficient neural networks? / F. Iandola [и др.] // arXiv preprint. — 2020. — arXiv: 2006.11316.
- [20] *Reimers N., Gurevych I.* Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks // Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP). — 2019. — С. 3982—3992.

- [21] *Smart Connections*. Smart Connections Plugin for Obsidian [Электронный ресурс]. — Режим доступа: <https://smartconnections.app/> (дата обращения: 06.05.2026).
- [22] *SkepticMystic*. Graph Analysis for Obsidian [Электронный ресурс]. — 2024. — Режим доступа: <https://github.com/SkepticMystic/graph-analysis> (дата обращения: 06.05.2026).
- [23] *Manning C. D., Raghavan P., Schütze H.* Introduction to Information Retrieval. — Cambridge University Press, 2008.
- [24] RAGAS: Automated evaluation of retrieval augmented generation / S. Es [и др.] // arXiv preprint. — 2023. — arXiv: 2309.15217.
- [25] spaCy: Industrial-strength Natural Language Processing in Python / M. Honnibal [и др.]. — 2020. — DOI: 10.5281/zenodo.1212303. — Режим доступа: <https://spacy.io/> (дата обращения: 10.05.2026).
- [26] *LanceDB*. LanceDB: Developer-friendly, serverless vector database. — Режим доступа: <https://lancedb.com/> (дата обращения: 25.02.2026).
- [27] *Reimers N., Gurevych I., Hugging Face*. Sentence-Transformers: State-of-the-Art Text Embeddings [Электронный ресурс]. — Режим доступа: <https://huggingface.co/sentence-transformers> (дата обращения: 10.05.2026).
- [28] *Gerganov G.* llama.cpp: Port of Facebook's LLaMA model in C/C++. — 2026. — Режим доступа: <https://github.com/ggerganov/llama.cpp> (дата обращения: 25.02.2026).
- [29] *Vasallo94*. ObsidianRAG: A Retrieval-Augmented Generation pipeline for Obsidian [Электронный ресурс]. — 2024. — Режим доступа: <https://github.com/Vasallo94/ObsidianRAG> (дата обращения: 06.05.2026).
- [30] *Beijing Academy of Artificial Intelligence (BAAI)*. bge-reranker-v2-m3 [Электронный ресурс]. — 2026. — Режим доступа: <https://huggingface.co/BAAI/bge-reranker-v2-m3> (дата обращения: 06.05.2026).

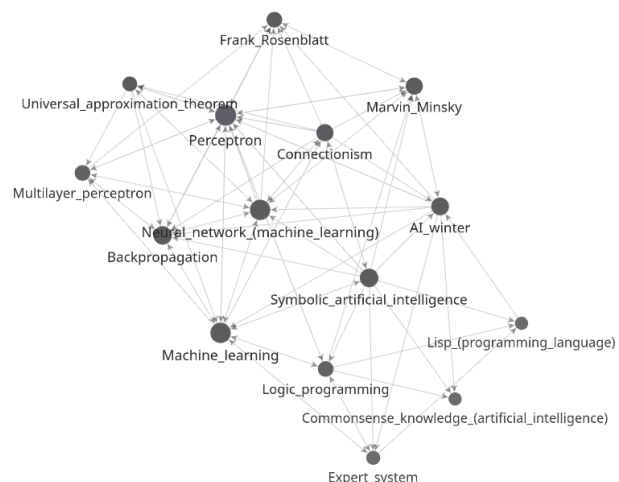
- [31] *noodleslove*. Personal Notes Repository [Электронный ресурс]. — 2026. — Режим доступа: <https://github.com/noodleslove/notes> (дата обращения: 06.05.2026).
- [32] *Hundred Rabbits*. 100r.co Source Code and Documentation [Электронный ресурс]. — 2026. — Режим доступа: <https://github.com/hundredrabbits/100r.co> (дата обращения: 06.05.2026).
- [33] *Smith L.* based.cooking [Электронный ресурс]. — 2026. — Режим доступа: <https://github.com/luke-smithxyz/based.cooking> (дата обращения: 06.05.2026).
- [34] *Akbary K.* Learning Notes [Электронный ресурс]. — 2026. — Режим доступа: <https://github.com/keyvanakbary/learning-notes> (дата обращения: 06.05.2026).
- [35] *Majlis M.* Wikipedia-API: Easy to use Python wrapper for Wikipedias' API [Электронный ресурс]. — Режим доступа: <https://github.com/martin-majlis/Wikipedia-API> (дата обращения: 10.05.2026).
- [36] *Sentence Transformers*. paraphrase-multilingual-MiniLM-L12-v2 [Электронный ресурс]. — 2026. — Режим доступа: <https://huggingface.co/sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2> (дата обращения: 06.05.2026).
- [37] *Beijing Academy of Artificial Intelligence (BAAI)*. BGE-M3 [Электронный ресурс]. — 2026. — Режим доступа: <https://huggingface.co/BAAI/bge-m3> (дата обращения: 06.05.2026).
- [38] *Jina AI*. jina-reranker-v2-base-multilingual [Электронный ресурс]. — 2026. — Режим доступа: <https://huggingface.co/jinaai/jina-reranker-v2-base-multilingual> (дата обращения: 06.05.2026).
- [39] *Alibaba Cloud*. Qwen 3 235B Instruct [Электронный ресурс]. — 2026. — Режим доступа: <https://huggingface.co/Qwen/Qwen3-235B-A22B-Instruct-2507> (дата обращения: 06.05.2026).

- [40] *Cerebras Systems*. Cerebras Inference API Documentation [Электронный ресурс]. — Режим доступа: <https://inference.cerebras.ai/> (дата обращения: 10.04.2026).
- [41] *Google DeepMind*. Gemma 3 27B: Open Models Based on Gemini Research and Technology [Электронный ресурс]. — 2026. — Режим доступа: <https://huggingface.co/google/gemma-3-27b-it> (дата обращения: 06.05.2026).
- [42] *Google DeepMind*. Gemma 4 31B: Advanced Open Models [Электронный ресурс]. — 2026. — Режим доступа: <https://huggingface.co/google/gemma-4-31b> (дата обращения: 06.05.2026).
- [43] *Google Cloud*. Vertex AI: Machine Learning Platform [Электронный ресурс]. — Режим доступа: <https://cloud.google.com/vertex-ai> (дата обращения: 10.04.2026).
- [44] *Google DeepMind*. Gemini 3 Flash: Fast and versatile multimodal model [Электронный ресурс]. — 2026. — Режим доступа: <https://ai.google.dev/gemini-api/docs/models/gemini> (дата обращения: 06.05.2026).
- [45] *conk7*. Knowledge Graph Builder [Электронный ресурс]. — 2026. — Режим доступа: https://github.com/conk7/knowledge_graph_builder (дата обращения: 11.05.2026).

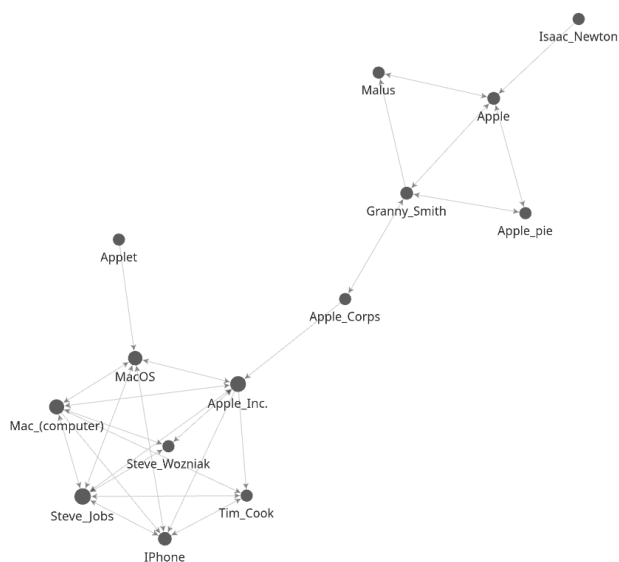
ПРИЛОЖЕНИЕ А. ВИЗУАЛИЗАЦИЯ ДАТАСЕТА



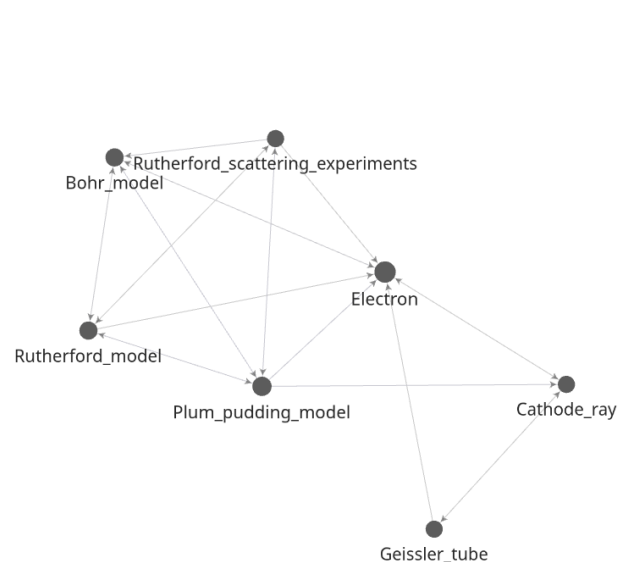
(a) Подкорпус Hub-and-Spoke



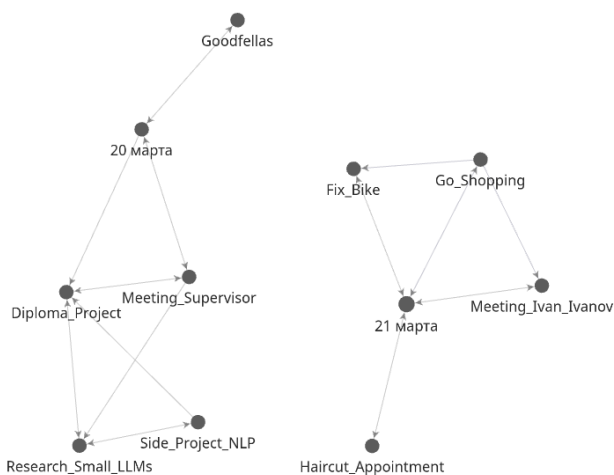
(b) Подкорпус Realistic



(c) Подкорпус Versus



(d) Подкорпус Sequence



(e) Подкорпус Personal

Рис. А.1 – Визуальные представления графов знаний подкорпусов датасета